

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

A REPORT

ON

RAG-POWERED ASSISTANT FOR INFRASTRUCTURE AS CODE GENERATION

BY

Name of the Student: Kolla Naga Manasa **ID No.:** 2024TM93519

AT

Caprus IT Private Limited, Hyderabad

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

November 2026

A REPORT

ON

RAG-POWERED ASSISTANT FOR INFRASTRUCTURE AS CODE GENERATION

BY

Name of the Student: Kolla Naga Manasa
ID No.: 2024TM93519
Discipline: M.Tech. (Software Engineering)

Prepared in partial fulfilment of the
WILP Dissertation course (BITS SEZG628T Dissertation)

AT

Caprus IT Private Limited, Hyderabad

ACKNOWLEDGEMENTS

I thank _____ for allowing this dissertation to be carried out at Caprus IT Private Limited and for the organisational support it needed.

I am grateful to my supervisor, Karthik Reddy Charla, for guidance throughout the project and for the freedom to pursue the questions the results raised, and to the additional examiner, Devoleena Das, for reviewing the work.

I thank my faculty mentor, Ashish Narang, for the mid-semester review, whose comments on policy retrieval, internal templates, retrieval experiments, failure analysis, user assessment and the structure of this report shaped the final phase of the work.

Finally, I thank the eight colleagues from the delivery team who gave their time to take part in the user evaluation.

November 2026

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI (RAJASTHAN)
WILP Division

Organization: Caprus IT Private Limited **Location:** Hyderabad

Duration: **Date of Start:** 25 July 2026

Date of Submission:

Title of the Project: RAG-Powered Assistant for Infrastructure as Code Generation

ID No./Name of the student: 2024TM93519 / Kolla Naga Manasa

Name(s) and Designation(s) of Supervisor and Additional Examiner: Karthik Reddy Charla, Technical Manager, Caprus IT Private Limited (Supervisor); Devoleena Das, Accenture Private Limited (Additional Examiner)

Name of the Faculty mentor: Ashish Narang

Key Words: Retrieval-Augmented Generation; Infrastructure as Code; Terraform; Google Cloud; Large Language Models; Policy as Code; Open Policy Agent; Developer Trust

Project Areas: Cloud Computing; DevOps; Software Engineering; Applied Artificial Intelligence

Abstract:

Writing Infrastructure as Code (IaC) by hand is slow and error-prone, and large language models (LLMs) can draft it but often ignore organisational conventions that a plain-language request never states. This dissertation builds and evaluates a Retrieval-Augmented Generation (RAG) assistant that turns natural-language requirements into Terraform for Google Cloud, grounded in provider documentation and organisational conventions, and checks every output with terraform validate and plan, Open Policy Agent policies and functional-intent checks.

At mid-semester (35 tasks, three arms, 525 samples), technical correctness was near ceiling for every arm, but overall pass was 57.0% without retrieval, 78.8% with RAG and 95.2% with the conventions pasted into the prompt, because the conventions were retrieved in only 12% of RAG samples.

The final evaluation (five arms, 875 samples, under all five policies) added a policy-pinning variant that always supplies the relevant policy pages. It reached 93.3%, against 50.9% without retrieval, 67.3% with RAG and 99.4% with pasted conventions, beating RAG on 10 tasks and losing none ($p = 0.002$). Remaining failures trace to vague policy text, a template defect and a faulty intent check. With eight engineers, the assistant roughly halved median task time ($p = 0.039$); usability was moderate (SUS 63.8).

Signature of Student	**Signature of Supervisor**
Name: Kolla Naga Manasa	Name: Karthik Reddy Charla
Date:	Date:

Table of Contents

ACKNOWLEDGEMENTS i

CHAPTER 1 – INTRODUCTION 1

1.1 Background and motivation 1

1.2 Objectives and their final status 1

1.3 Response to the mid-semester review 2

1.4 Organisation of this report 3

CHAPTER 2 – REVIEW OF LITERATURE 4

2.1 Retrieval-Augmented Generation 4

2.2 LLMs for infrastructure code and their benchmarks 4

2.3 Error taxonomies and knowledge injection 5

2.4 Verifier-based evaluation and policy compliance 5

2.5 Risks introduced by retrieval 6

2.6 Measuring usability and trust in automation 6

2.7 Gap addressed by this work 7

CHAPTER 3 – PROBLEM STATEMENT 8

3.1 Statement 8

3.2 Why the problem is not solved by retrieval alone 8

3.3 Scope, assumptions and constraints 8

CHAPTER 4 – METHODOLOGY USED 10

4.1 System Design 10
4.2 Implementation 24
4.3 Evaluation Methodology 26
CHAPTER 5 – RESULTS AND DISCUSSION 37
5.1 The final run 37
5.2 Headline metrics 38
5.3 Paired comparisons across tasks 39
5.4 Retrieving versus pinning organisational policy 40
5.5 Validation with the reviewed internal templates 42
5.6 Retrieval configuration experiments 43
5.7 Detailed failure analysis 44
5.8 By difficulty 49
5.9 Further checks from the earlier run 50
5.10 User evaluation: usability, completion time and developer trust 51
5.11 Discussion 54
CHAPTER 6 – CONCLUSION AND FUTURE SCOPE 56
6.1 Conclusion 56
6.2 Future scope 57
6.3 Plan of work as completed 58
APPENDIX A. TASK CATALOGUE 59

APPENDIX B. PROMPT TEMPLATES 61

APPENDIX C. ORGANISATIONAL POLICY RULES 63

APPENDIX D. RUN MANIFEST OF THE REPORTED BENCHMARK 64

REFERENCES 65

GLOSSARY 67

ABBREVIATIONS 68

List of Figures

Figure 1: System architecture: offline knowledge-base construction, online generation for the five experimental arms, and identical validation for all of them. 11

Figure 2: Use-case diagram of the RAG-IaC assistant 17

Figure 3: Component diagram showing the main components and their dependencies. 18

Figure 4: Class diagram of the pipeline and the generators. 19

Figure 5: Class diagram of the retrieval subsystem. 20

Figure 6: Class diagram of the validation and classification model. 21

Figure 7: Sequence diagram of a RAG request and its validation. 22

Figure 8: Activity diagram of the benchmark evaluation. 23

Figure 9: Deployment diagram of the prototype. 24

Figure 10: Pass rates by arm in the final run, with 95% confidence intervals. 38

Figure 11: Overall pass rate by task category in the final run. 41

Figure 12: Share of samples containing each error category in the final run. 45

List of Tables

Table 1: Objectives and their final status. 2

Table 2: Mid-semester review comments and the response to each. 2

Table 3: Technology choices and rationale. 12

Table 4: The four validation levels. 15

Table 5: Defects found in the tooling and their remedies. 26

Table 6: Task set composition. 28

Table 7: The five experimental arms. 29

Table 8: How the policy-pinning arm selects policy pages. 30

Table 9: Metrics. 30

Table 10: Findings of the technical review of the internal templates. 32

Table 11: User-evaluation design. 34

Table 12: Headline metrics of the final run (165 samples per arm). Proportions show the 95% Wilson interval in brackets. 38

Table 13: Paired comparisons of per-task overall pass rate (33 headline tasks; exact two-sided sign test). 39

Table 14: Tasks on which pinning the policy changed the overall pass rate by at least 40 percentage points. 39

Table 15: How the policy reached the model, and the result. * Scored against two of the five policies, so not comparable with the other rows. 40

Table 16: Overall pass rate by task category in the final run. 41

Table 17: Retrieval configurations of the RAG arm before the final run, scored against the earlier policy set. * Scored against all five policies, so its overall pass is not comparable with the other columns. 43
Table 18: The policy-pinning arm with different numbers of retrieved chunks, scored against all five policies. 44
Table 19: Failing samples by root cause in the final run (165 samples per arm). 44
Table 20: Node-pool sizes generated for gke-01, and which passed the intent check. 47
Table 21: Sensitivity of the overall pass rate to the gke-01 intent check. 48
Table 22: The two tasks that explicitly request policy-forbidden configurations (a public website bucket and a Cloud SQL instance with a public IP). 48
Table 23: Technical (non-policy, non-intent) diagnostics among the failing samples of the final run. 49
Table 24: Overall pass rate by difficulty level in the final run. 49
Table 25: User evaluation: task success and time to completion by condition and experience level. 51
Table 26: User evaluation results per task. 52
Table 27: Paired Wilcoxon signed-rank tests, one pair per participant (tied pairs dropped). A negative r means lower values with the assistant. 52
Table 28: Usability (SUS) and trust in the assistant, collected after the assistant condition. 52
Table 29: Plan of work and status. 58
Table 30: The 35 benchmark tasks. 60
Table 31: Policy rules. 63
Table 32: Glossary of technical terms. 67

Table 33: Abbreviations used in this report. 68

CHAPTER 1 – INTRODUCTION

1.1 Background and motivation

Infrastructure as Code (IaC) is now the standard way of provisioning and managing cloud resources: the desired infrastructure is described declaratively in version-controlled files, and a tool such as Terraform reconciles the real environment with that description. Writing correct IaC is nevertheless difficult. Provider schemas are large and change frequently (the Google Cloud provider used in this work was at major version 8 at the time of writing), resources depend on one another in ways that the documentation only describes indirectly, and every organisation layers its own conventions and security policies on top.

Large language models (LLMs) can turn a plain-English requirement into a plausible Terraform configuration, but plausibility is not correctness. A model's knowledge is frozen at training time, so it tends to produce arguments that have been renamed or removed, omits arguments that are now required, and knows nothing about the conventions of the organisation it is being used in. Retrieval-Augmented Generation (RAG) addresses exactly this weakness by placing retrieved, current documentation in the model's context at generation time instead of relying on what the model memorised.

The problem is directly relevant to the author's employer, Caprus IT, where delivery work provisions infrastructure largely through Terraform and where misconfiguration and the slow onboarding of new engineers continue to cost time. An assistant that converts plain-language requirements into validated, policy-compliant IaC would reduce authoring effort and lower the rate of misconfiguration-related incidents.

1.2 Objectives and their final status

The objectives set out in the approved outline, and how each was met, are:

#	Objective (from the outline)	Final status
O1	Study the state of the art in LLM-based code generation and RAG for IaC	Done (Chapter 2)
O2	Identify common LLM error categories in IaC generation and the knowledge gaps retrieval must close	Done: taxonomy applied to every sample and mapped onto a published one; detailed failure analysis (Section 5.7)
O3	Design a RAG architecture (ingestion, chunking, embedding, vector storage, retrieval, generation)	Done (Section 4.1), extended with per-resource policy indexing and policy pinning (Section 4.3.4)
O4	Build a working prototype that generates IaC from natural-language prompts	Done: runs end to end on Vertex AI (Sections 4.1, 4.2)
O5	Validate output with automated tooling and benchmark against a non-RAG baseline	Done: final benchmark of five arms and 875 samples, and seven retrieval configurations (Sections 5.1 to 5.6)
O6	User evaluation of usability and developer trust	Done with stated limitations: eight engineers, 32 task sessions (Section 5.10)

Table 1: Objectives and their final status.

1.3 Response to the mid-semester review

The mid-semester review asked for the evaluation to be strengthened in five specific ways and for the report to be restructured. Table 2 lists each request, what was done in response and where it is reported. Where a request could not be met in full, the table says so.

Review comment	What was done	Where
Improved retrieval of organisational policies	The policy was indexed per resource type (policy retrieval rose from 12% to 36% of samples), and a policy-pinning arm was built that always places the relevant policy pages in a mandatory prompt section	4.3.4, 5.4
Validation using reviewed internal templates	An independent review by the platform team was not available in time. The templates were given a documented technical self-review (nine findings, eight corrected) and the final run uses the corrected versions	4.3.7, 5.5
Additional retrieval configuration experiments	Seven configurations in all: number of chunks (3, 4, 8, 12), per-document cap removed, chunk size 800, and per-resource policy indexing	4.3.8, 5.6
Detailed failure analysis	Every failure classified by root cause and the configurations behind each cause compared across arms; six case studies, including a policy that does not say what is enforced, a template defect copied into code, and a defect found in the intent checker itself	5.7
User-level assessment of usability, completion time and trust	Counterbalanced study with eight engineers at two experience levels: SUS, a trust-in-automation scale and time to completion	4.3.9, 5.10
Report writing and chapter structure	Report reorganised into the six chapters requested and laid out to the WILP guideline	This report

Table 2: Mid-semester review comments and the response to each.

1.4 Organisation of this report

Chapter 2 reviews related work. Chapter 3 states the problem, with its scope, assumptions and constraints. Chapter 4 sets out the methodology: the system design and the reasons for each technology choice, the implementation, and the evaluation methodology, including the policy-pinning design, the technical review of the internal templates and the user-evaluation design. Chapter 5 presents and discusses the results of the final benchmark, the retrieval configuration experiments, the failure analysis and the user evaluation. Chapter 6 concludes and sets out future work. The appendices give the task catalogue, the prompts, the policy rules and the run manifest. A glossary of technical terms and a list of abbreviations follow the references.

CHAPTER 2 – REVIEW OF LITERATURE

2.1 Retrieval-Augmented Generation

Lewis et al. [1] introduced Retrieval-Augmented Generation as a way of combining a generator's parametric memory (knowledge stored in model weights) with a non-parametric memory: a dense index of documents that is queried at inference time and whose results condition the output. Two properties make the approach attractive for infrastructure code. The knowledge source can be updated without retraining the model, which matters when provider schemas change every few weeks, and the retrieved passages give generated output a traceable provenance. Khan et al. [8] report the practical side of building such systems, describing the end-to-end pipeline from data collection and preprocessing through retrieval indexing to response generation and the technical problems that arise at each stage. The present system follows the same staged decomposition.

2.2 LLMs for infrastructure code and their benchmarks

IaC-Eval [2] is, to our knowledge, the first dedicated benchmark for LLM-generated Terraform. It contains 458 human-curated scenarios and goes beyond syntax by checking whether a generated configuration satisfies an explicit intent specification. Its released tooling targets AWS, with other providers left as future work. This design, in which validity and intent are separate questions, is the model for the intent checks used in this work. Multi-IaC-Eval [5] extends evaluation across AWS CloudFormation, Terraform and CDK and reports that modern LLMs achieve success rates above 95% in generating syntactically valid IaC across formats, while semantic alignment and complex infrastructure patterns remain difficult. Taken together, these results show that syntactic validity is close to saturated for strong models and is therefore an insufficient measure of quality on its own.

2.3 Error taxonomies and knowledge injection

The closest prior work to this dissertation is that of Nekrasov et al. [3], who propose an error taxonomy for LLM-generated Terraform and enhance IaC-Eval with cloud emulation and automated error analysis. They compare knowledge-injection strategies that progress from naive RAG to Graph RAG with semantic enrichment and inter-resource dependency modelling. Their reported baseline LLM success is 27.1% overall; with knowledge injection, technical validation success rises to 75.3% and overall success to 62.6%. They identify a "Correctness-Congruence Gap": models become proficient coders but remain limited as architects in satisfying nuanced user intent. This is the reason the present evaluation reports well-formedness and intent satisfaction as separate quantities rather than a single score.

2.4 Verifier-based evaluation and policy compliance

Jouini [4] evaluates seven agentic strategies for Terraform generation on a 186-task benchmark using three verifier stages: terraform validate, terraform plan and OPA policy evaluation. Active retrieval raised the pass@1 of one small open model from 14.0% to 45.7% by reducing validation failures, and iterative refinement with verifier feedback reached 62.9% and 84.4% pass@1 for two further models. A finding that bears directly on this design is that 79% of the OPA failures remaining after refinement were information-gap failures that could be resolved when the policy text was visible to the model. That supports placing organisational conventions in the retrieval corpus, which is what this work does, and it motivates the same three verifier stages here.

Vargas et al. [6] benchmark seven proprietary and open models on secure AWS Terraform generation across 17 scenarios using Checkov and Trivy. They conclude that code validity and security are largely orthogonal properties and that prompt engineering alone is insufficient, so

automated scanning remains necessary. The consequence for the present study is that technical correctness and policy compliance are reported as distinct dimensions (Section 4.3.3) so that the effect of retrieval on each can be seen separately.

2.5 Risks introduced by retrieval

Retrieval also adds an attack surface. Ye et al. [7] describe ImportSnare, an attack in which poisoned documentation steers a retrieval-augmented code generator toward malicious dependencies; they report attack success rates above 50% for some popular libraries and success even at a poisoning ratio as low as 0.01%. Although their setting is package recommendation rather than infrastructure, the lesson transfers: whatever enters the knowledge base can shape the generated code. This work therefore restricts the corpus to a pinned release of the official provider documentation plus a small set of hand-written organisational conventions, and accepts no user-contributed content (Section 4.1.4).

2.6 Measuring usability and trust in automation

The System Usability Scale (SUS) [13] is a ten-item questionnaire, alternating positively and negatively worded statements on a five-point scale, that yields a single score from 0 to 100. It is short, technology-agnostic and widely used, which is why it has accumulated benchmarks: across a large body of published studies the mean score is about 68, so a score of 68 is commonly read as average usability [16]. A SUS score is not a percentage, and small samples give wide intervals around it.

Trust in automation is a distinct question from usability: an engineer can find a tool easy to use and still not rely on its output. Jian et al. [14] derived a 12-item scale empirically from how people describe trust and distrust in automated systems, rated on a seven-point scale. It is used here, adapted to code generation, because over-trust is the specific risk of an assistant

that writes infrastructure code: generated configuration that validates can still be insecure or non-compliant (Section 2.4), so the question is not only whether developers trust the output but whether that trust is warranted.

2.7 Gap addressed by this work

In the literature reviewed for this report, evaluation of LLM-generated Terraform is concentrated on AWS, and we did not find a GCP-specific benchmark. Existing studies also tend to report a single success figure; the combination of separate technical, intent and organisational-policy measurements, a per-sample error taxonomy, and a paired retrieval-versus-no-retrieval comparison on the same model and prompt is the contribution attempted here, for Google Cloud. The task set built for this purpose (Section 4.3.2) follows IaC-Eval's intent-specification approach but is a new, GCP-specific set of 35 tasks, so its results are not directly comparable to figures published for the AWS benchmarks.

CHAPTER 3 – PROBLEM STATEMENT

3.1 Statement

Given a natural-language infrastructure requirement, generate Terraform for Google Cloud Platform (GCP) that (a) is well-formed and consistent with the provider's schema, (b) actually implements what was asked, and (c) complies with organisational policy, which the requirement itself does not state. The central question is whether retrieval over provider documentation and organisational conventions improves these outcomes compared with the same model used without retrieval; where it does not, what kinds of error remain and why; and whether developers find such an assistant usable and trustworthy.

3.2 Why the problem is not solved by retrieval alone

Retrieval ranks documents by their similarity to the request. A request describes a resource ("a bucket that keeps old versions for 30 days"); organisational policy describes rules that apply to many resources ("every data store carries an environment label") and is rarely similar to any single request. The earlier run of this project (`main-01`) showed the consequence: the general policy document was retrieved in 12% of RAG samples, and the policy failures concentrated where it was absent. The problem addressed in the final phase is therefore how to get cross-cutting policy to the model reliably, and whether the model then follows it.

3.3 Scope, assumptions and constraints

The work designs, implements and evaluates a RAG-based assistant that accepts natural-language infrastructure requirements and produces Terraform for Google Cloud, grounded in a curated knowledge base of provider documentation and organisational conventions. Production deployment, multi-cloud coverage beyond GCP and continuous-integration rollout are outside the scope of this dissertation and are noted as future work. The assistant never applies infrastructure: generated code is only validated and planned.

The work assumes a single target model and provider version at a time (Section 4.3.3), that the organisation's conventions can be stated in a few short documents, and that automated validators (Terraform, OPA and a functional-intent check) are a sufficient proxy for correctness for the purposes of this study. The last assumption was tested in the failure analysis and found to hold with one exception, an intent check that rejected correct answers (Section 5.7.5). The organisational conventions, internal templates and policies were written by the author to stand in for a real organisation's documents, because the host organisation's own documents could not be used. The main constraints were a single workstation and API quota for all runs, and the availability of colleagues for the user evaluation.

CHAPTER 4 – METHODOLOGY USED

This chapter covers the system design and the reasons for each technology choice (4.1), the implementation and the engineering problems found and fixed along the way (4.2), and the evaluation methodology (4.3).

4.1 System Design

4.1.1 Design principles

Five principles guided the design, each chosen so that the eventual results can be trusted.

1. **A controlled comparison.** The two experimental arms use the same model, prompt template, target provider version and sampling settings. They differ in one respect only: whether retrieved context is supplied. Any difference in outcome can then be attributed to retrieval.
2. **Everything is measurable.** Each stage exposes its output so that failures can be attributed to retrieval, generation or validation rather than to the system as a whole.
3. **Safety by construction.** Generated code is only ever validated and planned, never applied, and planning runs against a dedicated experimentation project.
4. **Reproducibility.** Every run records the model, temperature, index size, tool versions, source-document release and hashes of the task file and policies, and stores every generated configuration.
5. **Provenance.** The knowledge base contains only public provider documentation, pinned to a release, and hand-written organisational conventions; no customer or production configuration is used.

4.1.2 Architecture

Figure 1 shows the pipeline. An offline stage builds the knowledge base: documents are cleaned, split into chunks, embedded and stored in a vector index. At run time a requirement is either passed through the retriever (RAG arm) or straight to the model (baseline arm); the generated Terraform then passes through the same validation and evaluation stages regardless of arm. The other three arms change only the context: the conventions and templates arms paste documents in place of retrieved chunks, and the policy-pinning arm adds the selected policy pages to the retrieved chunks (Section 4.3.4).

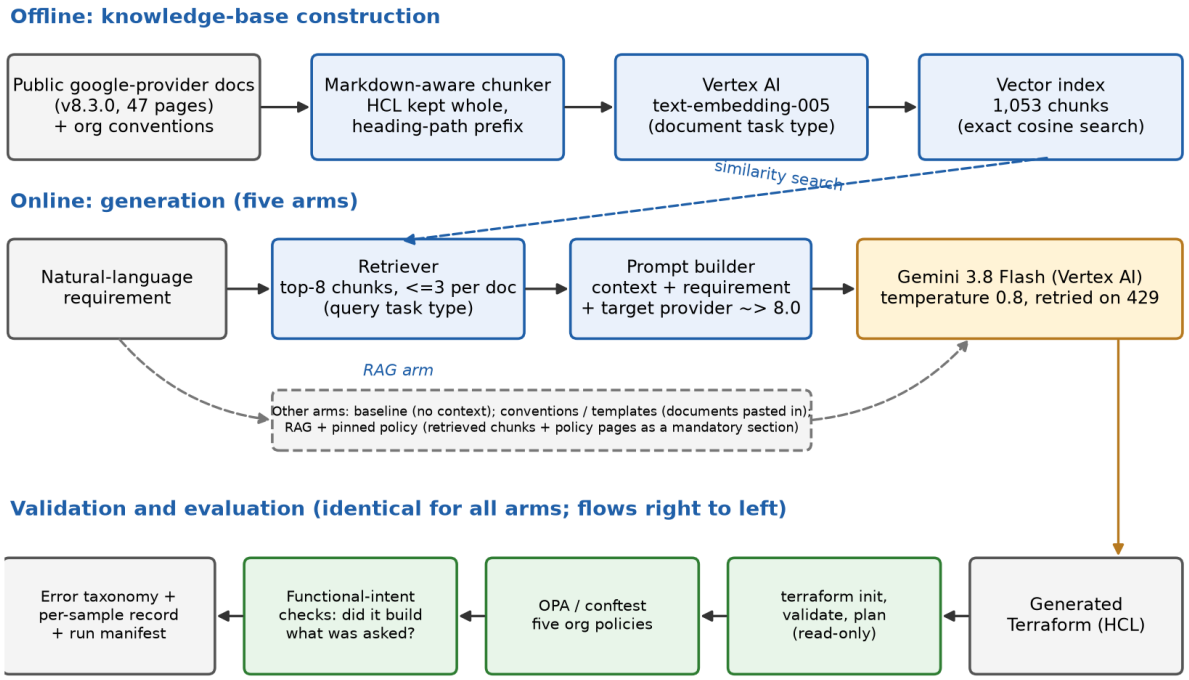


Figure 1: System architecture: offline knowledge-base construction, online generation for the five experimental arms, and identical validation for all of them.

4.1.3 Technology choices

Table 2 lists the choices that the supervisor asked to be settled and stated at this stage, with the reasoning for each.

Concern	Choice	Rationale
Cloud and IaC tool	Google Cloud Platform; Terraform with the `google` provider (`~> 8.0`)	Terraform is the delivery team's tool; GCP is the cloud used for all experimentation.
Region	`asia-south1` (Mumbai) for storage and embeddings	Google Cloud has no region in Hyderabad. Of its two Indian regions, Mumbai is roughly 600 km away versus roughly 1,350 km for Delhi, and no cost reason to prefer Delhi was identified, so proximity decided it.
Generation model	Gemini 3.8 Flash on Vertex AI, global endpoint	The current general-availability Flash model. `gemini-2.0-flash` no longer exists on Vertex AI and `gemini-2.5-flash` is scheduled to shut down on 20 October 2026, inside this project's timeline. The global endpoint avoids losing access to newer models that are not yet regional.
Embeddings	Vertex AI `text-embedding-005` (768 dimensions)	Supports separate document and query task types, which the model is trained to expect for asymmetric retrieval.
Vector store	In-memory exact cosine search, persisted to disk	See Section 4.1.3.1.
Policy engine	Open Policy Agent via Conftest 0.70.0 (OPA 1.20.2)	Policy-as-code checks named in the outline; Rego rules are versioned with the code.
Validation	Terraform 1.16.2: init, validate, plan	Standard tool-level verification; plan is read-only.

Table 3: Technology choices and rationale.

4.1.3.1 Vector store decision

The knowledge base is about 1,000 chunks (roughly 1 MB of text). At that size an exact cosine search over an in-memory matrix returns in well under a millisecond and needs no infrastructure. Vertex AI Vector Search is an approximate-nearest-neighbour service designed for very large corpora and is billed while an index endpoint is deployed, so at this scale it would add cost and operational overhead without improving retrieval quality. The vector store sits behind a small interface and a Vertex AI Vector Search implementation is already written, so moving to it is a configuration change if the corpus grows substantially. The decision is therefore an in-memory store for the dissertation, with a documented migration path.

4.1.4 Knowledge base

The knowledge base combines two sources. The first is 47 pages of public documentation for the `google` Terraform provider, taken from release v8.3.0 of the official repository (MPL-2.0

licence). The pages cover the resources most often needed for common infrastructure: storage, networking, compute, databases, Kubernetes, serverless, messaging, analytics, security, and platform services such as Artifact Registry and DNS. Each page is cleaned by removing generated front matter and the import and timeout sections, which add noise without helping generation; a comment recording the source URL, release and licence is added to the top.

The second source is a set of four short documents written for this project that state organisational conventions in the form a practitioner would document them: the default region, mandatory `environment` labels, no public exposure unless explicitly requested, deletion protection and backups for production data stores, and regional over zonal configurations for production. They stand in for the sanitised internal templates the supervisor asked to be used. A `templates` arm compares them with simulated internal templates written in a messier, wiki-like style, which were technically reviewed before the final run (Section 4.3.7). Three further short documents restate the policy for each resource type it covers (Cloud Storage, Cloud SQL, GKE), so that policy competes with resource-specific documentation in retrieval (Section 4.3.4).

Because retrieved text shapes generated code, the corpus is treated as a trust boundary (Section 2.5). It is fetched over HTTPS from the official repository at a pinned tag, is regenerated by a script rather than edited by hand, and contains no customer, production or credential data.

4.1.5 Chunking and retrieval

Provider documentation is long: a single resource page can run to tens of thousands of characters, and its most valuable content is a fenced HCL example whose lines must stay

together. A plain paragraph splitter would cut examples at blank lines. The chunker therefore keeps fenced code blocks whole, splits oversized blocks only at line boundaries and re-fences each piece, and prefixes every chunk with its heading path (for example, `[google_storage_bucket > Argument Reference]`) so that a chunk from the middle of a long argument list still identifies the resource it describes. The final corpus yields 1,053 chunks with a 1,500-character target.

Queries are embedded with the retrieval-query task type and chunks with the retrieval-document type. The retriever returns the eight most similar chunks with at most three from any single document. The cap was added after inspection showed that a long page could fill every slot: for a requirement asking for a VPC with subnets and Cloud NAT, the top eight results came entirely from the NAT and subnetwork pages and none described the network resource itself. Even with the cap, retrieval remains imperfect, which is why retrieval quality is measured explicitly (Section 4.3.4) rather than assumed.

4.1.6 Generation

The system prompt tells the model it is generating Terraform for GCP, to treat retrieved documentation as authoritative in preference to its own memory, to say so in a comment when the retrieved context does not cover something rather than guessing, to include provider and required-provider blocks, and to follow secure defaults. The baseline prompt is identical except that it omits the retrieval instructions and states that no reference material is provided. Both prompts name the target provider version constraint, because otherwise a model that recalls an older provider from memory would be validated against a different schema from the one the retrieved documents describe, confounding the comparison.

Model replies are post-processed to extract the Terraform: models often wrap code in a Markdown fence and sometimes add commentary, and newer models can return several content parts. Transient quota errors are retried with exponential backoff and jitter.

4.1.7 Validation stack

Generated code is checked at three levels, each answering a different question (Table 3).

Level	Tool	Question answered	Example failure it catches
Well-formed	`terraform init` and `validate`	Is the code syntactically valid and consistent with the provider schema?	An argument that does not exist in provider v8
Plannable	`terraform plan` (read-only)	Can Terraform build an execution plan against the real API?	An invalid combination of values rejected by the provider
Policy-compliant	OPA through Conftest	Does it obey organisational policy?	A bucket with no `environment` label; an `allUsers` IAM grant; a firewall open to 0.0.0.0/0
Intent-satisfying	Task-specific intent checks	Did it build what was asked?	A valid, compliant configuration for the wrong resource, or with the wrong retention period

Table 4: The four validation levels.

Three details matter for the credibility of the measurements. First, plan failures caused by the environment (missing credentials) or by a variable declared without a default are recorded but not counted against the model, since they say nothing about the code. Second, when the policy engine cannot parse a configuration at all, that is recorded as a syntax failure and not as a policy violation. Third, the policy rules were verified against known-violating and known-compliant configurations, because policy code can fail silently (Section 4.2.4).

4.1.8 Safety

The system never runs `terraform apply`. Plan runs only against a dedicated experimentation project and is read-only. Credentials are Application Default Credentials held locally, and no secrets are stored in the repository.

4.1.9 UML Design Views

This section presents the design as UML diagrams. They were drawn from the code base, so the classes, methods and call sequences shown are those implemented, not an idealised design.

4.1.9.1 Use-case view

Two people use the system. A developer requests Terraform for a natural-language requirement and reviews the validation report; this use case includes retrieving documentation context (in the RAG arm), generating Terraform with the LLM, and validating it with terraform and the policy checks. A researcher builds the knowledge-base index, runs the benchmark over the five arms, and summarises and compares the results. Re-running samples lost to infrastructure failure extends the benchmark. Vertex AI, Terraform and Conftest are external systems on which the assistant depends.

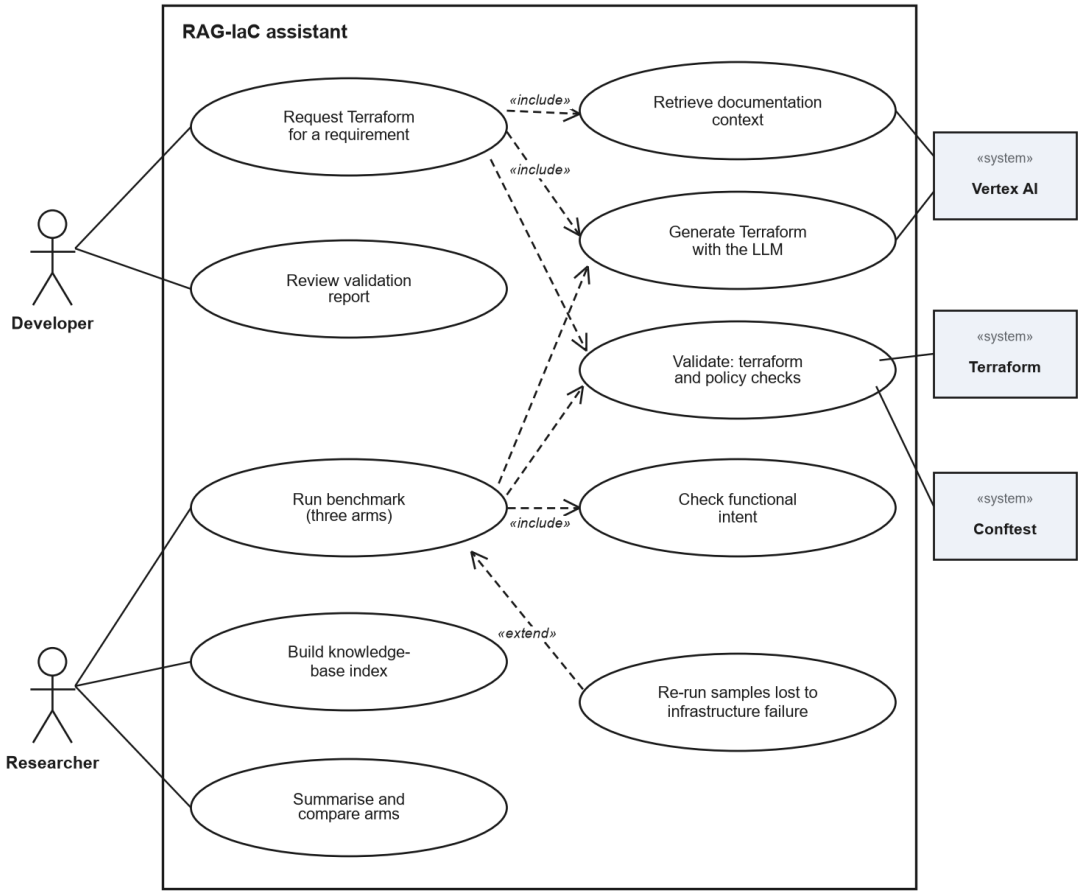


Figure 2: Use-case diagram of the RAG-IaC assistant.

4.1.9.2 Component view

The Pipeline class is a facade over ingestion, retrieval, generation and validation. The Streamlit UI and the command-line scripts call it, and the evaluation package calls it to run the benchmark. Embeddings and the vector store sit behind interfaces so that an implementation can be replaced without changing their callers. External dependencies are Vertex AI for embeddings and Gemini, the terraform and conftest programs run as local processes, and local files for the knowledge base, the vector index and the results. The Vertex AI Vector Search store is implemented but not provisioned (Section 4.1.3).

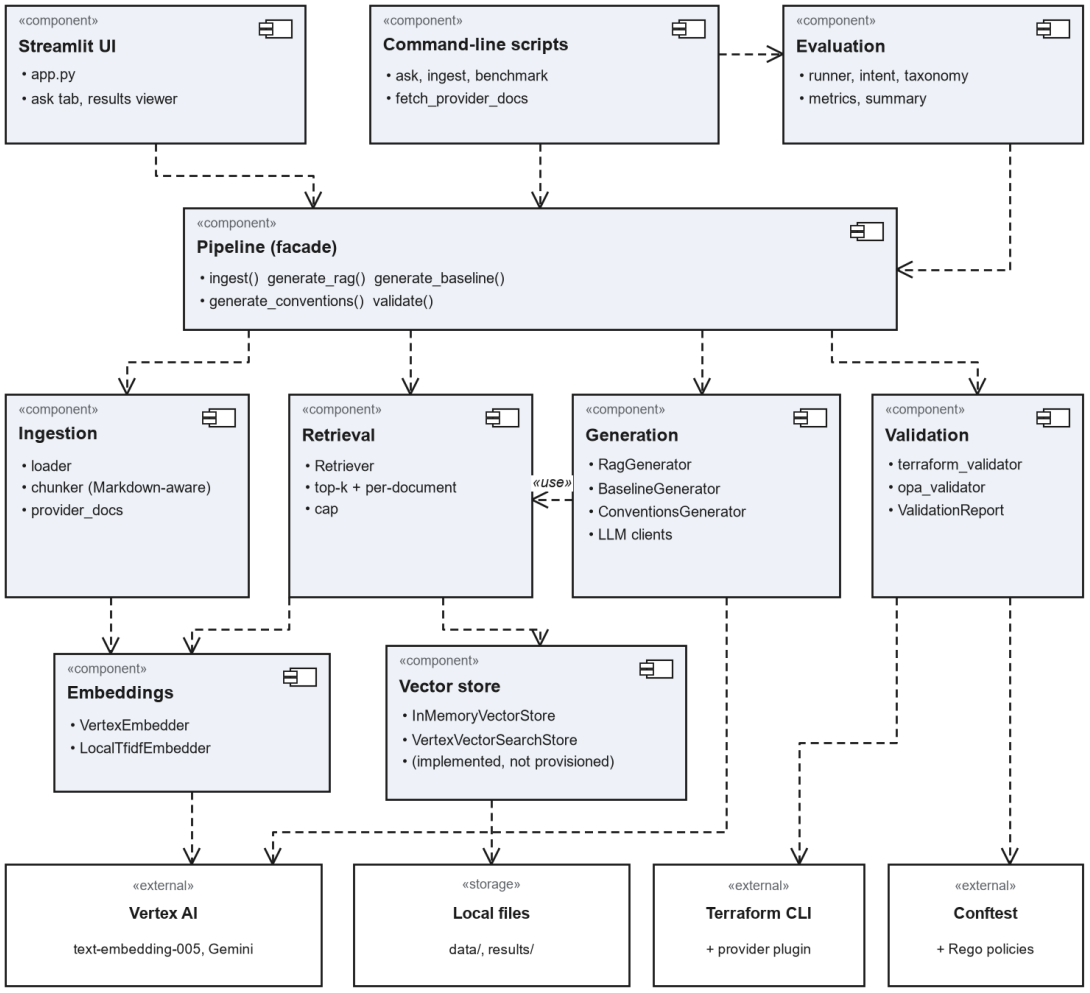


Figure 3: Component diagram showing the main components and their dependencies.

4.1.9.3 Class views

Figure 4 shows the classes that generate Terraform. Pipeline holds the embedder, vector store, retriever and the three generators, and exposes ingest, the three generate methods and validate. All three generators use one LLM interface and differ only in the context they supply: retrieved chunks, all convention documents, or none. LLM is a structural interface (a Python Protocol) with a Gemini implementation and a rule-based implementation used only for offline development. The policy-pinning generator added for the final run (Section 4.3.4) is a fourth generator of the same shape: it holds the retriever and the LLM, and adds the knowledge-base

directory it reads policy pages from. It is not drawn in the diagram, which shows the generators as built for the earlier run.

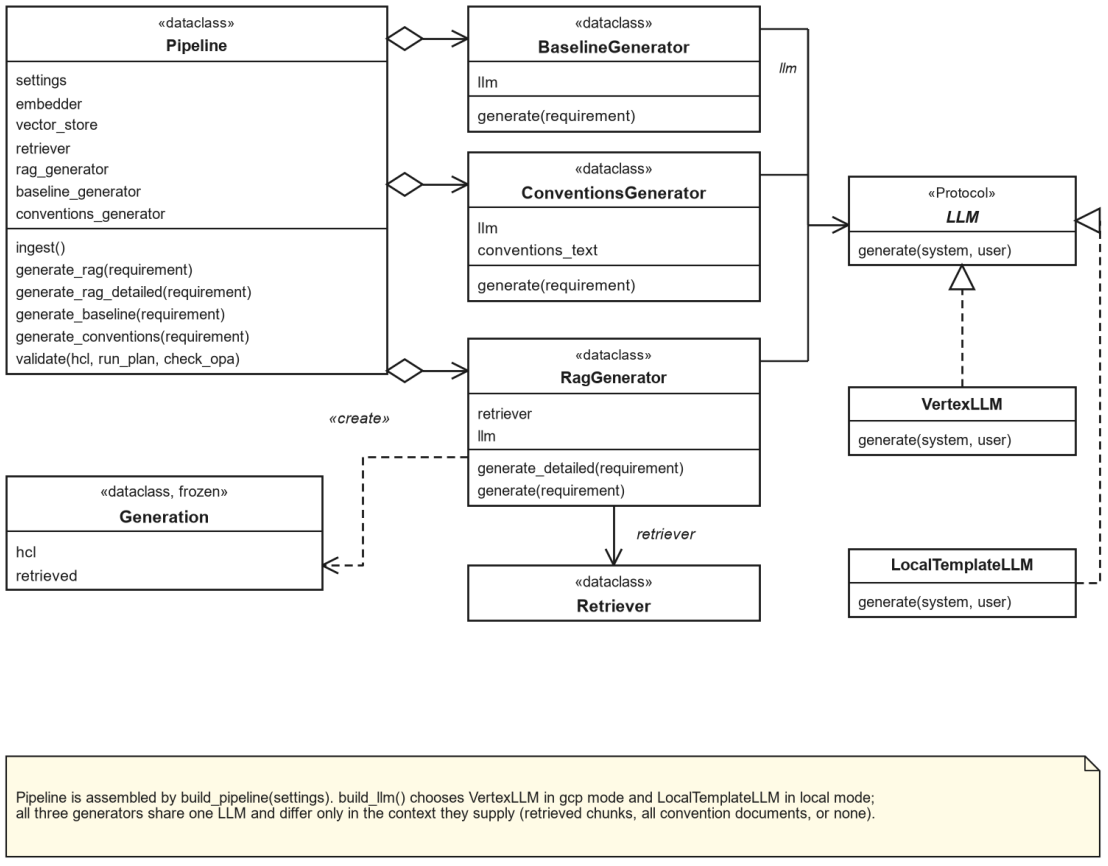


Figure 4: Class diagram of the pipeline and the generators.

Figure 5 shows the retrieval subsystem. The Retriever depends on the Embedder and VectorStore interfaces. The experiments use the Vertex AI embedder and the in-memory store; the TF-IDF embedder supports offline development and the Vertex AI Vector Search store supports a larger corpus. Search returns scored chunks, each holding one chunk of a source document.

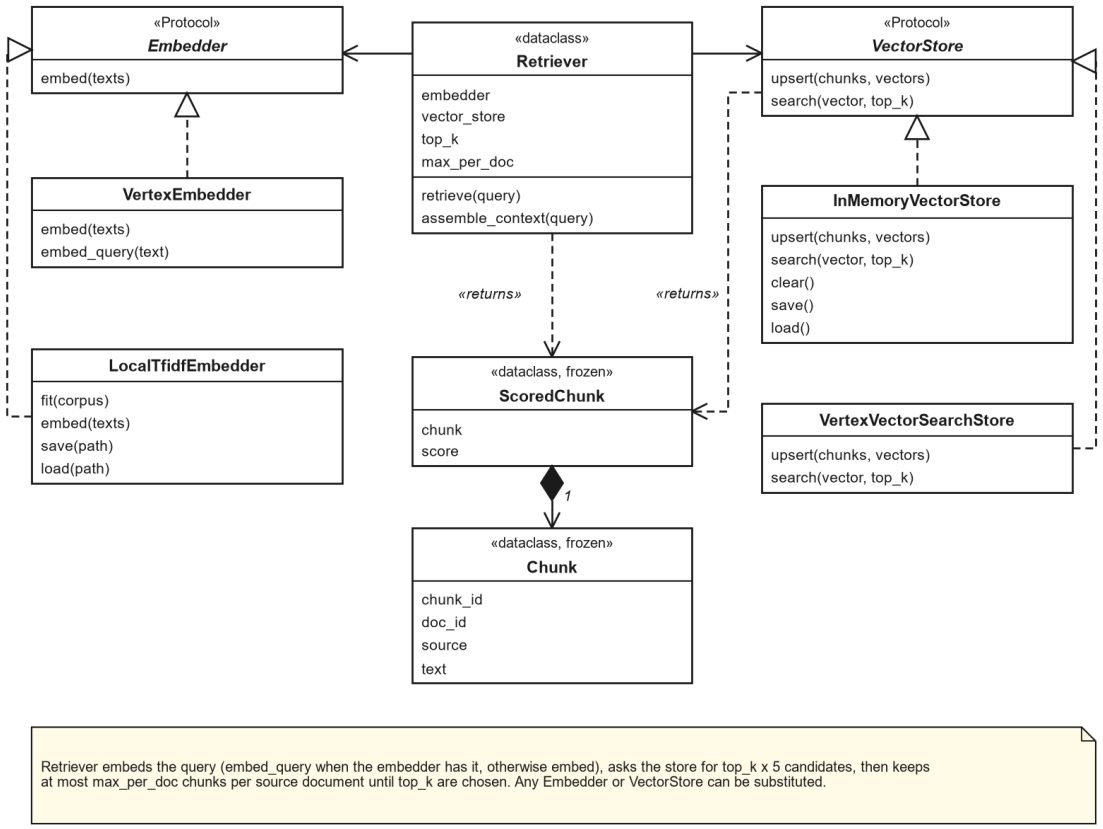


Figure 5: Class diagram of the retrieval subsystem.

Figure 6 shows the validation and classification data model. A ValidationReport holds one result per terraform stage and an optional policy result. The taxonomy maps the validator outputs and the intent result onto error categories. The policy result records whether conftest could evaluate the input, so that HCL that cannot be parsed is counted as a syntax error and not as a policy violation.

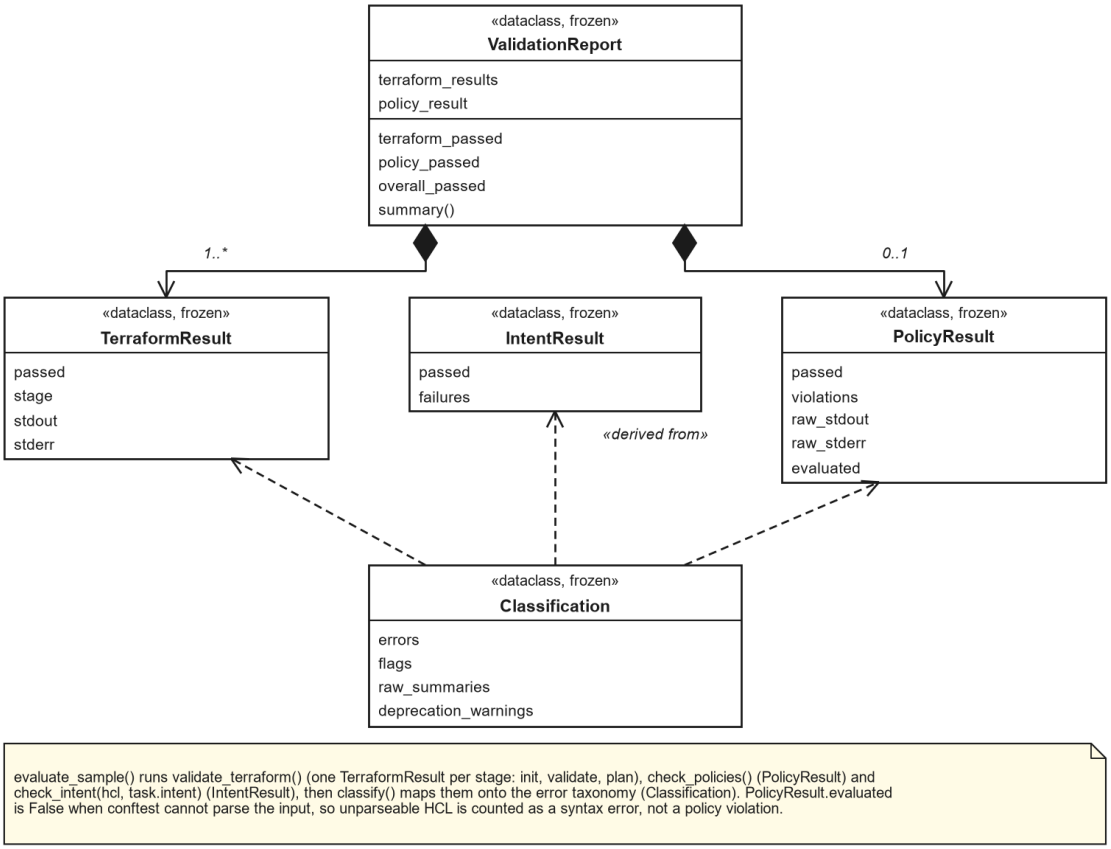


Figure 6: Class diagram of the validation and classification model.

4.1.9.4 Sequence view

Figure 7 follows one RAG request. The retriever embeds the requirement and asks the vector store for more candidates than needed (five times top-k) when a per-document cap is set, then applies the cap to choose the top-k chunks. The RagGenerator sends the prompt and context to Gemini and returns the Terraform together with the retrieved chunks. Validation is a separate step: the pipeline runs terraform init, validate and plan, then the conftest policy check, and returns a ValidationReport. Embedding and generation calls retry with exponential backoff on rate-limit and availability errors.

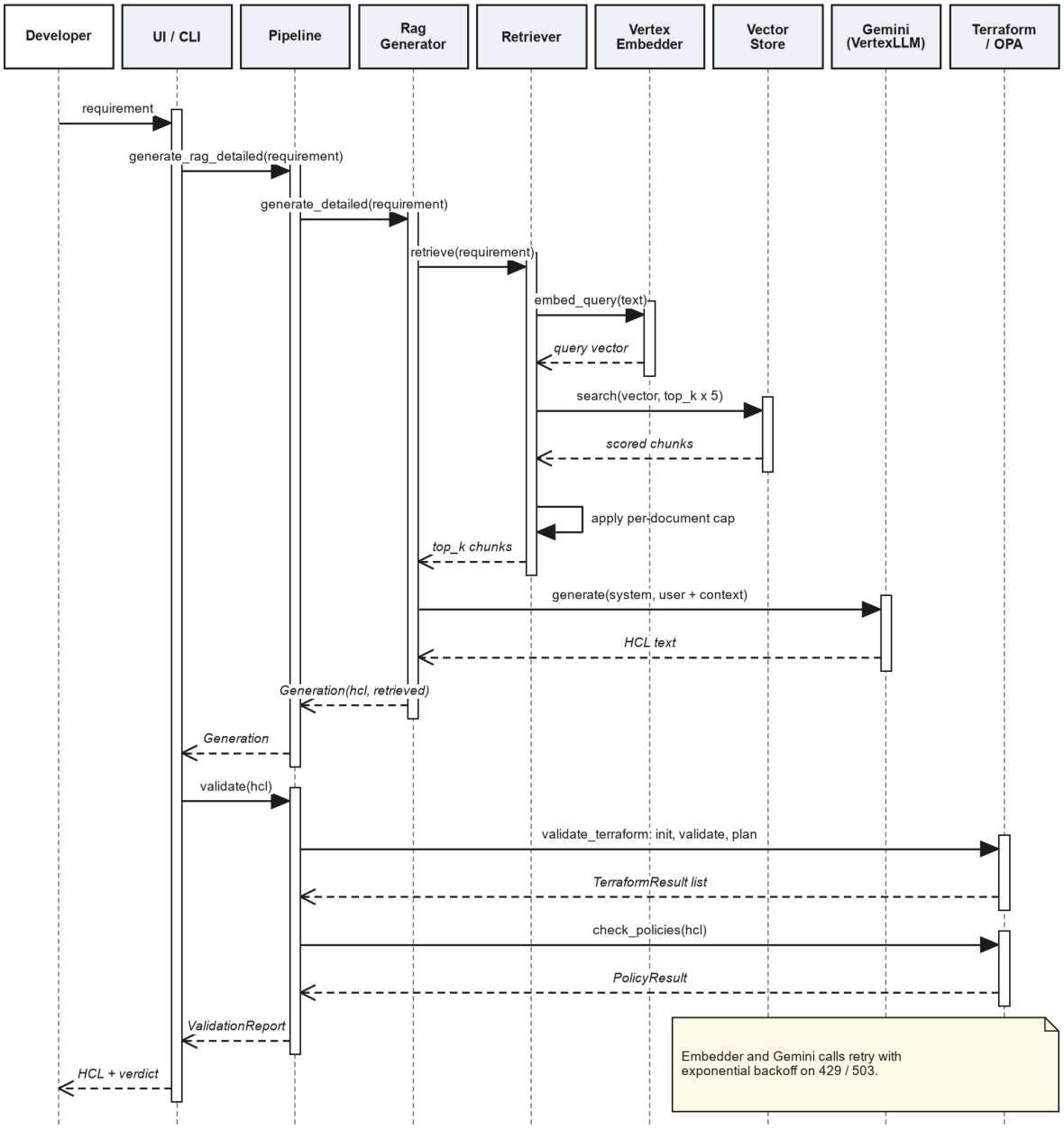


Figure 7: Sequence diagram of a RAG request and its validation.

4.1.9.5 Activity view

Figure 8 shows what the benchmark does for each sample. Samples already recorded are skipped, which makes a run resumable. The arm selects how the Terraform is generated; a failed generation is recorded as a result, not a crash. Terraform, policy and intent checks

follow, and errors are classified. A sample lost to infrastructure failure is flagged and not scored, and is re-run later. When all samples are done, the run is summarised.

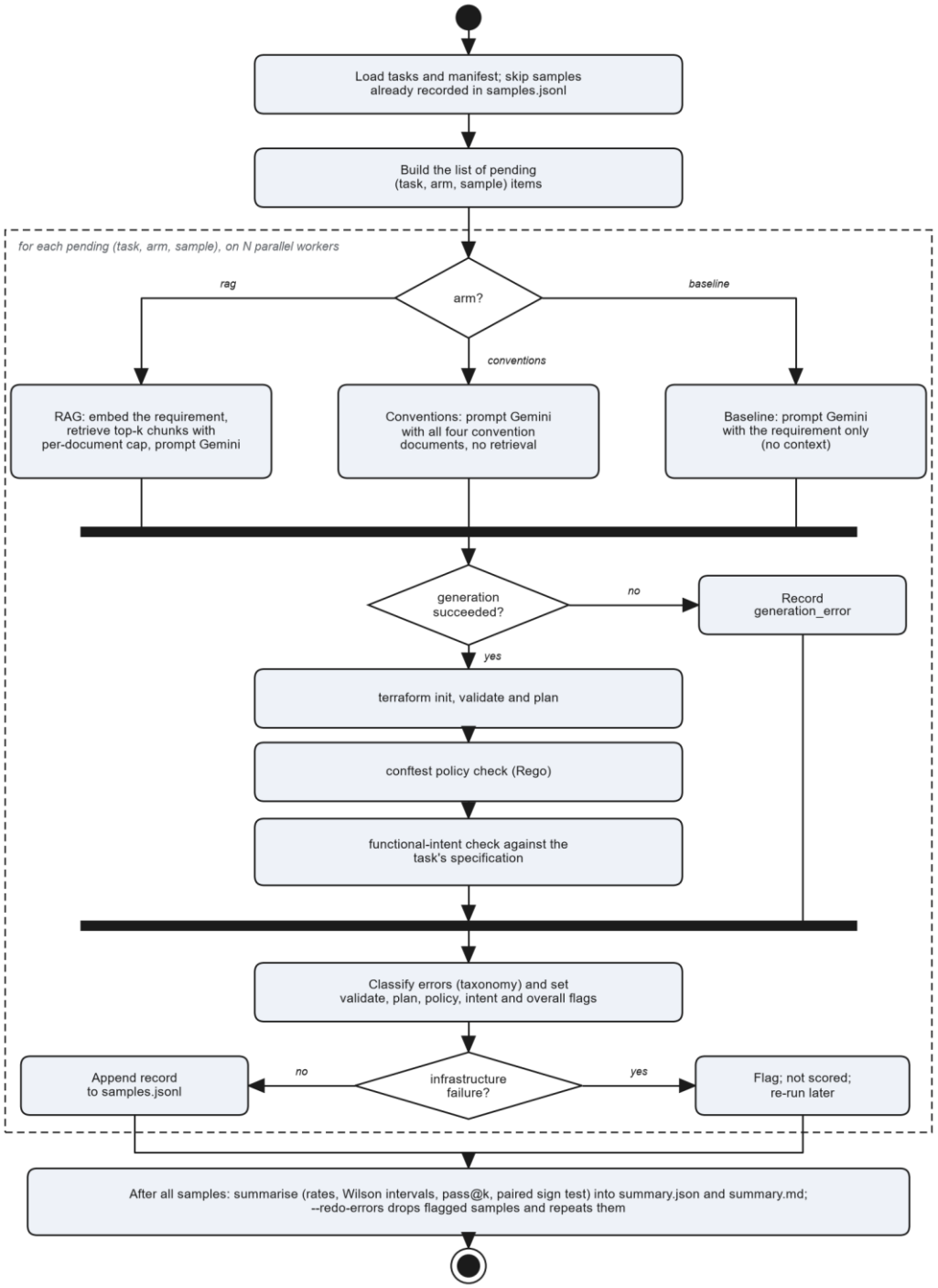


Figure 8: Activity diagram of the benchmark evaluation.

4.1.9.6 Deployment view

The prototype runs on one developer workstation: a Python environment with the pipeline and the Streamlit UI, terraform and conftest as local programs, and the knowledge base, vector index and results as files. It calls Vertex AI over HTTPS with Application Default Credentials for embeddings and generation. Terraform downloads the Google provider from the Terraform Registry and caches it. The provider documentation was fetched once from GitHub. Terraform plan makes read-only calls to Google Cloud and never applies.

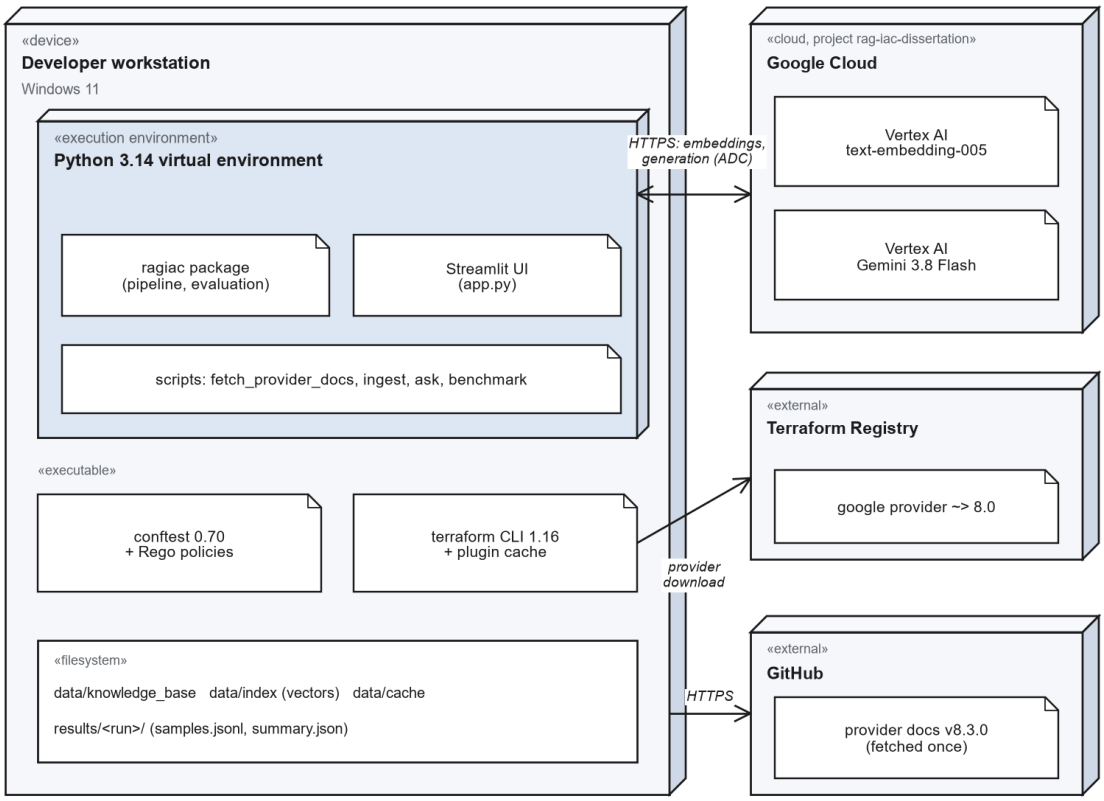


Figure 9: Deployment diagram of the prototype.

4.2 Implementation

4.2.1 Code base

The prototype is a Python package organised by pipeline stage: ingestion, embeddings, vector store, retrieval, generation, validation and evaluation. Every external dependency (Vertex AI,

terraform, conftest) sits behind a small interface, which allows the full pipeline to run offline in a development mode and lets unit tests substitute fakes. A browser interface built with Streamlit lets a user submit a requirement and see the retrieved context, the generated Terraform and the validation results side by side, and browse stored benchmark runs. The code base has 146 automated tests, including integration tests that run the real policy engine.

A rule-based template generator exists so that the pipeline can be exercised without cloud credentials. It is not a language model, is clearly labelled as such in its output, and none of the results in this report come from it.

4.2.2 Reproducibility

Each benchmark run writes a manifest recording the git commit, model, sampling temperature, embedding model, chunking and retrieval settings, index size, provider-documentation release, target provider constraint, Terraform and Conftest versions, and hashes of the task file and the policy files. Every generated configuration, its retrieved chunks and the full validator diagnostics are stored per sample, and all summary statistics are computed from those stored records, so a table in this report can be regenerated or audited from the raw data. Runs are resumable, and samples lost to infrastructure failure can be re-run without touching the rest.

4.2.3 Local development mode

For offline development the pipeline substitutes a TF-IDF embedder, the in-memory vector store and the rule-based template generator. This mode is used for testing the plumbing only.

4.2.4 Engineering problems found and fixed

Building the evaluation infrastructure produced a number of defects, several of which would have silently distorted results had they not been found. They are recorded here because they

show why the measurement tooling needed its own testing, and because they are candidate entries for the error analysis of tool-assisted generation.

Problem	Effect if unnoticed	Fix
Model reply wrapped in a Markdown fence	`terraform init` fails on every sample, so every sample would count as a syntax error	Extract the HCL from fenced replies, including replies with surrounding prose or a truncated closing fence
Newer model returns several content parts	Text accessor raises; samples lost or mis-scored	Join text parts, skip reasoning parts; found during the first full run
Model identifier retired (`gemini-2.0-flash`)	All generation fails with a 404	Moved to the current GA model and recorded the shutdown date of the alternative
Retry caught the wrong exception class for HTTP 429	No retries; ingestion and generation fail under quota limits	Catch the REST transport's exception; regression test added
Policy rules written for an older Rego syntax and ignoring the parser's list wrapping	One rule denied every resource, another never fired, so public resources would have passed	Rules rewritten and tested against known-bad and known-good configurations
Re-ingestion appended instead of replacing chunks	Duplicate chunks skew retrieval	Idempotent upsert by chunk identifier
Retrieval hit measured by filename	Under-reports retrieval quality, because several files describe one resource	Match on the resource types the retrieved text actually mentions
Variables without defaults counted as plan failures	Idiomatic code penalised for something plan cannot evaluate	Classified as not evaluable, alongside credential errors

Table 5: Defects found in the tooling and their remedies.

4.3 Evaluation Methodology

The metrics, the baseline, RAG and conventions arms and the analysis described here were fixed before the earlier benchmark run (`main-01`), as the supervisor required. The templates arm was added to that run later. The policy-pinning arm, the template review and the additional retrieval experiments were added after the mid-semester review, in response to its comments and to what the earlier run showed. They were evaluated on freshly generated samples in the final run (`final-01`), but they were designed with knowledge of the earlier results, which is stated as a threat to validity in Section 4.3.11.

4.3.1 Research questions

6. **RQ1 (technical correctness).** Does retrieval increase the proportion of generated configurations that validate, plan and satisfy the stated intent?
7. **RQ2 (organisational policy).** Does retrieval increase conformance to organisational policy, which the requirement never states, and is any gain due to retrieval or simply to the conventions being available to the model?
8. **RQ3 (errors).** Which error categories dominate, what causes them, and how does retrieval change their distribution?
9. **RQ4 (retrieval).** How often does retrieval surface the documentation a task needs, including the policy, and which retrieval configuration works best?
10. **RQ5 (users).** Do developers complete tasks faster or more successfully with the assistant, and how usable and trustworthy do they find it?

4.3.2 Task set

The published IaC benchmarks reviewed target AWS, so a GCP task set was built for this work. It contains 35 natural-language requirements: 33 headline tasks and 2 tasks that deliberately conflict with organisational policy. Difficulty is graded 1 to 3: level 1 is a single resource with one or two constraints (10 tasks), level 2 a single resource with several interacting constraints (12 tasks), and level 3 several related resources (13 tasks). Table 6 summarises the categories; the full list is in Appendix A.

Category	Tasks	Coverage
storage	6	Cloud Storage buckets: versioning, lifecycle rules, access settings, IAM
database	5	Cloud SQL: engine and tier, availability, backups, private IP, users and databases
kubernetes	4	GKE clusters and node pools: private nodes, Workload Identity, Autopilot, autoscaling
network	6	VPC, subnets, firewall rules, Cloud Router and NAT, global addresses, private DNS
compute	2	Compute Engine instances with specific machine, zone, image and service-account settings
serverless	3	Cloud Run, including IAM restricted to one invoker; Cloud Scheduler
messaging	2	Pub/Sub topics and subscriptions, including dead-letter policy
data	2	BigQuery datasets and tables
security	2	Secret Manager and Cloud KMS with rotation
platform	1	Artifact Registry
policy_conflict	2	Requirements that explicitly ask for something organisational policy forbids

Table 6: Task set composition.

Each task carries a machine-checkable intent specification in the style of IaC-Eval [2]: the resource types that must be present and assertions on their attributes (for example, that versioning is enabled and a lifecycle rule moves objects to NEARLINE after 30 days). Specifications allow alternatives where a requirement can legitimately be met in more than one way (an IAM member or an IAM binding; `node_count` or `initial_node_count`) and evaluate constant arithmetic, so a correct answer is not rejected for its form. The two policy-conflict tasks (a publicly readable website bucket and a Cloud SQL instance with a public IP) are reported separately: a model that does what was asked will violate policy by design, and mixing them into the headline would confuse the comparison.

4.3.3 Experimental arms and sampling

Five arms are compared, all using the same model, temperature, target provider constraint and task set. They differ only in the context the model is given:

Arm	Context given to the model	Question it answers
Baseline	None	What the model does from its own knowledge
RAG	The eight chunks most similar to the requirement, at most three per document, from provider documentation and organisational policy	What retrieval adds
Conventions in prompt	The four hand-written convention documents pasted in whole; no retrieval and no provider documentation	What knowing the conventions adds, without retrieval
Simulated internal templates	As above, but with the four wiki-style internal templates in place of the conventions	Whether the conventions result survives messier documentation
RAG + pinned policy	The RAG context, plus the general policy and the policy page for each resource type retrieved, in a mandatory section of the prompt	Whether policy reaches the model and is followed when it is not left to ranking

Table 7: The five experimental arms.

Every task is run five times per arm at temperature 0.8, so that the samples differ and pass@k is meaningful. All five arms of the final run were generated interleaved in one session, from one commit and one index, so drift in the hosted model between arms is not a concern within the run. The retrieval configuration experiments use three samples per task, so their pass@1 and pass@3 are the figures that can be compared with the main run.

4.3.4 Retrieving and pinning organisational policy

The earlier run showed that policy was retrieved too rarely (Section 3.2). Two changes were made in response. First, the policy was **indexed per resource type**: three short documents were added to the knowledge base that restate the policy for Cloud Storage, Cloud SQL and GKE in that resource's own vocabulary, so that each competes for retrieval with the provider pages of the same resource rather than one general page competing with all of them. This is part of the index for every retrieving arm in the final run.

Second, a **policy-pinning** arm (`rag_policy`, `src/ragiac/generation/policy_pinning.py`) takes policy out of the ranking altogether. It retrieves exactly as the RAG arm does, then selects policy pages by rule rather than by similarity: the general policy (`org_best_practices.md`) always, and the resource-specific policy page for every resource type whose provider page

appears among the retrieved chunks (Table 8). The selected pages are placed whole in a separate prompt section headed "Organisational policy (mandatory)", any retrieved chunks from those pages are dropped from the ordinary context to avoid duplication, and the system prompt adds one sentence stating that the policy section is mandatory unless the requirement explicitly asks for something different (Appendix B.4).

Retrieved provider page	Policy page pinned
`storage_bucket`, `storage_bucket_iam`, `storage_bucket_object`	`gcs_bucket_policy.md`
`sql_database_instance`, `sql_database`, `sql_user`	`cloud_sql_policy.md`
`container_cluster`, `container_node_pool`	`gke_cluster_policy.md`
(any request)	`org_best_practices.md`

Table 8: How the policy-pinning arm selects policy pages.

The design keeps the prompt short: at most two policy pages are pinned for a single-resource request, instead of pasting every convention document as the conventions arm does, so it would still work for an organisation whose conventions are too long to paste whole. Its weakness is that it relies on a hand-maintained mapping, so a resource type with no policy page of its own (BigQuery, here) gets only the general policy. Section 5.7.3 shows the effect of that.

4.3.5 Metrics

Metric	Definition
Validate pass rate	Proportion of samples for which `terraform init` and `terraform validate` succeed.
Plan pass rate	Proportion for which `terraform plan` succeeds, among samples where plan could be evaluated (credential and unset-variable failures excluded).
OPA violation rate	Proportion of samples that violate at least one policy rule, among samples the policy engine could parse.
Intent pass rate	Proportion of samples that satisfy the task's intent specification.
Functional pass rate	Validates, plans where evaluable, and satisfies intent: technical correctness independent of organisational policy.
Overall pass rate	Functional pass and no policy violation.
pass@k	Unbiased estimator [9] of the probability that at least one of k samples is overall-correct, averaged over tasks (k = 1, 3, 5).
Retrieval hit rate	Fraction of the resource types a task needs that appear in the retrieved chunks (RAG arm only).

Table 9: Metrics.

Reporting functional and overall pass rates separately is deliberate. Following the observation in [6] that validity and security are largely orthogonal, and the intent-versus-correctness gap in [3], a single score would hide whether a change helps the model write correct infrastructure, helps it follow organisational conventions, or both.

4.3.6 Error taxonomy, retrieval quality and failure analysis

Every failed sample is mapped to error categories derived from what the validators report, grouped by the stage that catches them: generation failure; provider initialisation; HCL syntax; schema errors (unsupported argument or block, missing required argument, invalid value, conflicting arguments, invalid reference, expression error); plan errors; policy violations (missing labels, public access, other policy); and intent failures (missing resource, attribute mismatch). Messages that fit no category fall into an *other* bucket with the raw diagnostic kept. The taxonomy is mapped onto the published taxonomy of Nekrasov et al. [3] (Section 5.7.7).

Two retrieval measures are used. The **hit rate** is the share of the resource types a task needs whose provider documentation appears among the retrieved chunks. The **policy retrieval rate** is the share of samples in which at least one policy document was among the retrieved chunks. The first measures whether the model was given the right reference material, the second whether it was given the rules.

The detailed failure analysis goes beyond the counts. Every failing sample of the final run was assigned to a root cause from its diagnostics. For each root cause, the generated configurations were then read and compared across all arms on the affected tasks, to find what differed between passing and failing samples. Where a root cause pointed at the evaluation itself (an intent check or a policy rule), its effect is reported as a sensitivity analysis next to the pre-specified result, which is never changed after the fact.

4.3.7 Technical review of the internal templates

The `templates` arm uses four internal-template documents written in the informal style of a team wiki (`data/internal_templates/`). They were written by the author to simulate real internal documentation; they are not exports of the organisation's own documents. An independent review by the organisation's platform team was requested but was not available within the project timeline, so the templates were given a documented technical self-review instead. The full review record is `docs/internal_templates_review.md`; this section summarises it.

Method. Every HCL example in the templates was extracted and run through `terraform init` and `terraform validate` (Terraform 1.16.2, Google provider v8.9.0) and through `conftest` against all five organisational policies. Every written rule was then checked against the pinned provider documentation (v8.3.0), against the other templates and against the Rego policies. Where a defect could affect generated code, the stored benchmark samples were searched for its effect.

#	Document	Severity	Finding	Correction
F1	Cloud SQL	High	Point-in-time recovery advised via `point_in_time_recovery_enabled` for every engine; it is valid only for PostgreSQL and SQL Server (MySQL uses `binary_log_enabled`).	Rule split by engine
F2	Cloud Storage	High	Both lifecycle-rule snippets put two arguments in a single-line block, which is invalid HCL (`terraform init` fails).	Multi-line blocks
F3	Cloud SQL	Medium	Example disables the public IP without a `private_network`, which the API rejects at apply time (passes validate, plan and OPA).	`private_network` added
F4	GKE	Medium	Node pool runs as the Compute Engine default service account, commonly granted project Editor, contrary to the IAM rule.	Dedicated service account
F5	GKE	Low	`node_count` on a regional pool is per zone; the example's 2 creates 6 nodes, which is not stated.	Stated; example corrected
F6	GKE	Low	`remove_default_node_pool` is deprecated in provider v8.9 for new GKE versions.	Note added
F7	Platform standards	Low	Header claimed an export from a real wiki with a named owner; the document is simulated.	Replaced by a provenance note
F8	Platform standards	Low	"Every resource needs labels" is wider than the label policy enforces.	Scope stated
F9	Platform standards	Info	Regional pricing claim not verified.	Left as written

Table 10: Findings of the technical review of the internal templates.

Effect on generated code. Finding F1 is visible in the benchmark. On task `sql-02` (a highly available MySQL instance with backups) in run `main-01`, the samples that failed `terraform plan` with the error *"point_in_time_recovery_enabled is only available for the following [POSTGRES SQLSERVER]"* were: templates 4 of 5; conventions 4 of 5; rag 0 of 5; baseline 2 of 5. The hand-written conventions given to the `conventions` arm contained the same incorrect advice. The baseline, given no documents, shows that the model makes this mistake on its own some of the time; the flawed advice made it the norm. In every `rag` sample for this task, retrieval returned both the flawed conventions page and the provider page for `google_sql_database_instance`, whose MySQL examples use `binary_log_enabled`; the model followed the provider page each time. This is a direct case of a defect in an organisational document being copied into generated infrastructure code when it is the only guidance the model has, and it is one of the failure cases analysed in Section 5.7.4.

Corrections and their use. All findings except F9 were corrected in the templates. F1 and F3 were corrected in the hand-written conventions as well, because they contained the same two defects. After correction, every template example passes `terraform validate` and all policy checks. The final benchmark run (`final-01`, Section 5) uses the corrected templates and conventions. This review checks technical correctness and consistency with the enforced policies. It does not show that the templates match the organisation's actual practice. That needs an independent review by the platform team, which is listed under Future Scope.

4.3.8 Retrieval configuration experiments

Retrieval settings were varied one at a time, holding everything else fixed. Before the mid-semester review, the RAG arm was run with three chunks instead of eight, without the per-document cap, with an 800-character chunk size (in a separately embedded index) and with

the per-resource policy documents added. Those runs were scored against the earlier policy set, so their overall pass rates are comparable with each other and with `main-01` but not with the final run. For the final phase, the policy-pinning arm was run again with four and with twelve chunks, to check whether its result depends on the amount of retrieved context once policy no longer has to win a place in it. All configuration runs use three samples per task.

4.3.9 User evaluation design

The user evaluation measures usability, time to completion and developer trust (objective O6). The design below was set out in the mid-semester report and run as described, except where Section 5.10 states otherwise. The harness (instrument scoring, counterbalanced assignment, session recording and paired analysis) is in `src/ragiac/evaluation/user_study.py`.

Element	Design
Participants	8 engineers from the delivery team, voluntary: 4 with under one year of Terraform experience and 4 with two or more years.
Design	Within-subjects and counterbalanced. Each participant completes four tasks: two with the assistant and two without it, using provider documentation and search as they normally would. Condition order alternates across participants.
Tasks	Four tasks from the benchmark set (`gcs-01`, `gke-01`, `net-03`, `sql-03`), spanning difficulty levels 1 to 3. No customer or production information is used.
Objective measures	Task success (the benchmark's automated validity and intent checks) and time to completion. The number of edits to the assistant's output was planned but not recorded.
Instruments	System Usability Scale (10 items) [13] and a 12-item trust-in-automation scale [14], adapted to code generation, after the assistant condition; five semi-structured interview questions.
Analysis	Exact Wilcoxon signed-rank tests on per-participant means, with rank-biserial effect sizes; SUS and trust scored in the standard way.
Ethics	Informed consent, anonymised data, organisational permission, and the option to withdraw at any time.

Table 11: User-evaluation design.

4.3.10 Statistical treatment

Proportions are reported with 95% Wilson score intervals [10], which behave sensibly at small sample sizes and at 0% and 100%. Because the same 33 headline tasks are run under every arm, arms are compared per task: for each task the mean overall pass rate of each arm is computed, and the number of tasks on which each arm is better is tested with an exact two-

sided sign test. Five such comparisons are reported for the final run, so a single p-value near 0.05 should be read with that in mind. In the user evaluation, each participant contributes one pair of per-condition means to an exact Wilcoxon signed-rank test.

4.3.11 Threats to validity

- **Own task set, specifications and policies.** Tasks, intent checks, conventions and policies were written by the author. The failure analysis found one intent check that rejected correct answers (Section 5.7.5); others may exist that were not exposed by any arm's output.
- **Designed after seeing the earlier results.** The policy-pinning arm and the per-resource policy documents were designed in response to the earlier run, on the same task set. They were evaluated on new samples, but the task set is not independent of the design.
- **Simulated organisational documents.** The conventions, internal templates and policies stand in for a real organisation's. The template review checked their technical correctness, not whether they match real practice (Section 4.3.7).
- **Static intent checking.** Intent is checked on the parsed HCL; values computed through variables or `for_each` cannot be resolved, so tasks were written to avoid needing that.
- **One model, one temperature.** Results are for Gemini 3.8 Flash at temperature 0.8 (with a temperature 0 check in the earlier run) and should not be generalised to other models.
- **Sample size.** 33 headline tasks with five samples each give usable but wide intervals; category-level differences are indicative.
- **User evaluation.** Eight participants, a manual condition that some participants ran with a general-purpose AI assistant, and only two counterbalancing sequences (Section 5.10).

CHAPTER 5 – RESULTS AND DISCUSSION

This chapter reports the final benchmark run, `final-01` (Sections 5.1 to 5.3), then takes the mid-semester review's requests in turn: policy retrieval and pinning (5.4), the reviewed internal templates (5.5), the retrieval configuration experiments (5.6), the detailed failure analysis (5.7) and the user evaluation (5.10). Results from the earlier run, `main-01`, are used only for comparison and are labelled as such.

5.1 The final run

The final run evaluates 33 headline tasks and 2 policy-conflict tasks under 5 arms with 5 samples per task per arm (875 samples), using `gemini-3.8-flash` at temperature 0.8, `terraform plan` enabled and an index of 1053 chunks. It started on 2026-11-03 and ran in one session. No sample was lost to infrastructure failure, so every sample was scored. The full settings are in Appendix D.

Four things changed between the earlier run and this one, and they make the two runs' overall pass rates not directly comparable:

- **All five organisational policies are enforced.** The earlier run was scored against the label and public-access rules only; broad service-account roles, declared retention or deletion protection, and hardcoded secrets are now checked as well (Appendix C).
- **The policy is indexed per resource type.** The three resource-specific policy documents (Section 4.3.4) are in the index for every retrieving arm.
- **The internal templates and the Cloud SQL conventions were corrected** after the technical review (Section 4.3.7).
- **A fifth arm, RAG with pinned policy, was added** (Section 4.3.4).

5.2 Headline metrics

Metric	Baseline	Conventions in prompt	RAG	Simulated internal templates	RAG + pinned policy
Validate pass (init + validate)	98.8% [96-100]	100.0% [98-100]	98.8% [96-100]	99.4% [97-100]	99.4% [97-100]
Plan pass (where evaluable)	100.0% [97-100]	100.0% [98-100]	100.0% [98-100]	100.0% [98-100]	100.0% [98-100]
Intent pass	97.6% [94-99]	100.0% [98-100]	100.0% [98-100]	97.0% [93-99]	99.4% [97-100]
Functional pass (validate + plan + intent)	97.0% [93-99]	100.0% [98-100]	98.8% [96-100]	96.4% [92-98]	99.4% [97-100]
OPA violation rate (of evaluable)	48.2% [41-56]	0.6% [0-3]	32.1% [25-40]	9.7% [6-15]	6.1% [3-11]
Overall pass (functional + policy)	50.9% [43-58]	99.4% [97-100]	67.3% [60-74]	86.7% [81-91]	93.3% [88-96]
pass@1 (overall)	50.9%	99.4%	67.3%	86.7%	93.3%
pass@3 (overall)	51.5%	100.0%	72.7%	93.9%	93.9%
pass@5 (overall)	51.5%	100.0%	75.8%	97.0%	93.9%
Retrieval hit rate	-	-	97.0%	-	97.0%

Table 12: Headline metrics of the final run (165 samples per arm). Proportions show the 95% Wilson interval in brackets.

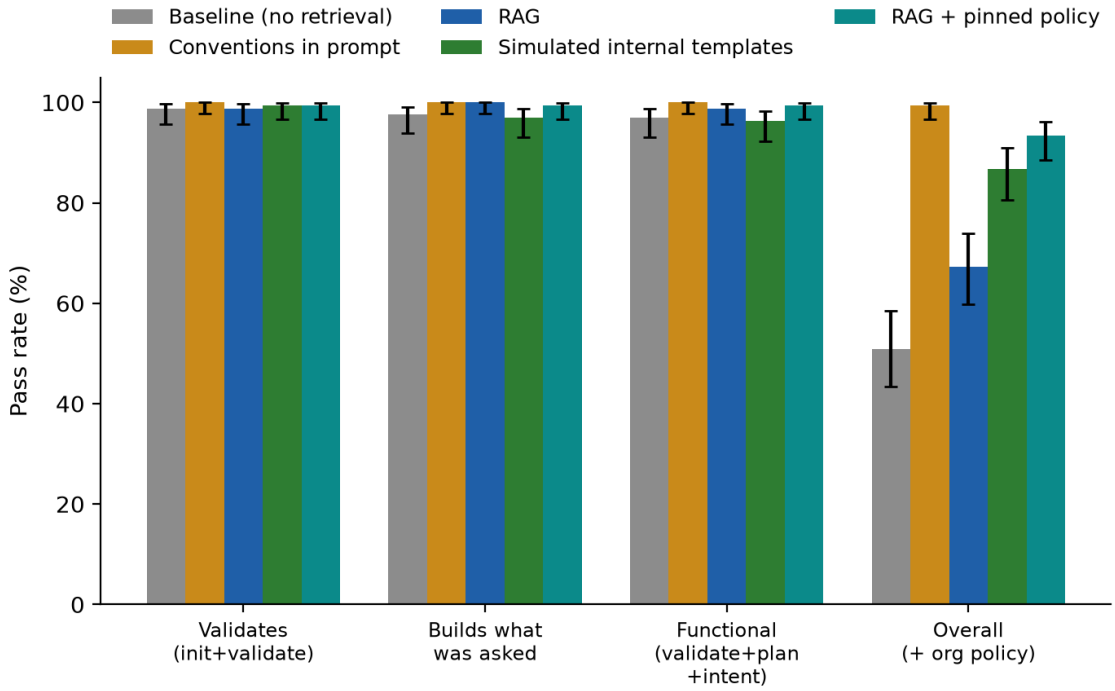


Figure 10: Pass rates by arm in the final run, with 95% confidence intervals.

Technical correctness is near ceiling in every arm. Functional pass rates, which require validate, plan and intent to pass, range from 96.4% to 100.0%, with overlapping intervals. No arm has more than six functional failures in 165 samples, so this run, like the earlier one, gives no evidence that retrieval improves technical correctness (RQ1).

Policy compliance separates the arms. Overall pass, which adds the five policy checks, is 50.9% for the baseline, 67.3% for RAG, 86.7% for the simulated internal templates, 93.3% for RAG with pinned policy and 99.4% for the conventions in the prompt. The baseline's pass@5 is 51.5%, hardly above its pass@1: its failures are systematic omissions, not bad luck on a given sample. RAG rises from 67.3% at pass@1 to 75.8% at pass@5, so for some tasks it complies on some samples and not others, which is what one would expect if the outcome depends on whether policy happened to be retrieved.

5.3 Paired comparisons across tasks

First arm	Second arm	First better	Second better	Tied	Mean difference	Sign test p
RAG	Baseline	9	0	24	+16.4 pts	0.0039
RAG	Conventions in prompt	0	12	21	-32.1 pts	0.0005
Conventions in prompt	Simulated internal templates	7	0	26	+12.7 pts	0.0156
RAG + pinned policy	RAG	10	0	23	+26.1 pts	0.0020
RAG + pinned policy	Conventions in prompt	0	3	30	-6.1 pts	0.2500

Table 13: Paired comparisons of per-task overall pass rate (33 headline tasks; exact two-sided sign test).

Task	Requirement	Pinned minus RAG
gcs-03	A Cloud Storage bucket with uniform bucket-level access enabled and public access prevention enforced.	+100 pts
sql-02	A highly available (regional) MySQL 8.0 Cloud SQL instance with automated backups enabled.	+100 pts
sql-03	A PostgreSQL Cloud SQL instance with no public IP address and deletion protection enabled.	+100 pts
sql-04	A PostgreSQL Cloud SQL instance with a database named appdb and a user named appuser.	+100 pts
sql-05	A Cloud SQL PostgreSQL instance with point-in-time recovery enabled.	+100 pts
gke-02	A private GKE cluster (private nodes) with Workload Identity enabled.	+100 pts
gcs-04	A Cloud Storage bucket in europe-west1 using the COLDLINE storage class.	+80 pts
sql-01	A PostgreSQL 15 Cloud SQL instance using the db-custom-2-8192 machine tier.	+80 pts
gcs-01	A Cloud Storage bucket named acme-invoices-archive in asia-south1 with object versioning enabled.	+60 pts
gcs-06	A private Cloud Storage bucket and a service account that has read-only object access (roles/storage.objectViewer) to that bucket.	+40 pts

Table 14: Tasks on which pinning the policy changed the overall pass rate by at least 40 percentage points.

Treating each task as one observation gives the same ordering as the pooled rates. Pinning the policy is better than RAG on 10 of 33 tasks and worse on 0 ($p = 0.002$), and it is the largest paired difference in the run. The tasks it fixes are the Cloud SQL, Cloud Storage and GKE

tasks, the resource types for which a policy page exists. RAG remains better than the baseline on 9 tasks and worse on none. The conventions in the prompt are better than pinned policy on 3 tasks and worse on none, a difference that is not significant (p = 0.25); two of them are the BigQuery tasks, for which there is no policy page to pin (Section 5.7.3). The conventions are better than the simulated templates on 7 tasks and worse on none (p = 0.016), discussed in Section 5.5.

5.4 Retrieving versus pinning organisational policy

Arm (run)	Policy in the model's context	Samples missing a label	Overall pass
RAG, general policy only (earlier run)	20 of 165 (12%)	33 of 165 (20%)	78.8% *
RAG, policy indexed per resource	60 of 165 (36%)	33 of 165 (20%)	67.3%
RAG + pinned policy	165 of 165 (100%), pinned	0 of 165 (0%)	93.3%
Conventions in prompt	165 of 165 (100%), pasted	0 of 165 (0%)	99.4%

Table 15: How the policy reached the model, and the result. * Scored against two of the five policies, so not comparable with the other rows.

Indexing the policy per resource type made it retrievable. With one general policy document, the policy was among the retrieved chunks in 20 of 165 (12%) RAG samples in the earlier run, from 4 tasks. With the three resource-specific documents added, it was retrieved in 60 of 165 (36%), from 12 tasks. That is the improvement the review asked for, but it did not translate into compliance. In the final run, RAG samples that retrieved a policy document passed overall in 37 of 60 (62%) cases, against 74 of 105 (70%) for samples that did not, and 14 of the samples that retrieved a policy document still left out the label. (Policy documents are retrieved mainly for the label-bearing resource types, where failing is possible at all, so the two groups are not like for like. But retrieving the policy clearly did not guarantee following it.) In the earlier run, every one of the few samples that retrieved the policy complied with it. The difference is that a retrieved chunk is one passage among eight, often a fragment of the policy page, presented as reference material.

Pinning the policy made it followed. The policy-pinning arm supplied the general policy in every sample and a resource-specific policy page as well in 100 of 165 (61%) (`cloud_sql_policy.md` in 45, `gcs_bucket_policy.md` in 35, `gke_cluster_policy.md` in 20). Not one of its samples was missing a label, against 33 RAG samples, and its overall pass rate is 26.1 percentage points above RAG's. It reached this with at most two policy pages per request, instead of every convention document. Table 16 shows where the difference lies.

Category	Baseline	Conventions in prompt	RAG	Simulated internal templates	RAG + pinned policy
compute	100%	100%	100%	100%	100%
data	0%	90%	0%	30%	0%
database	0%	100%	4%	100%	100%
kubernetes	25%	100%	75%	75%	100%
messaging	90%	100%	100%	100%	100%
network	100%	100%	100%	97%	100%
platform	100%	100%	100%	100%	100%
security	100%	100%	100%	100%	100%
serverless	100%	100%	100%	100%	100%
storage	0%	100%	50%	70%	97%

Table 16: Overall pass rate by task category in the final run.

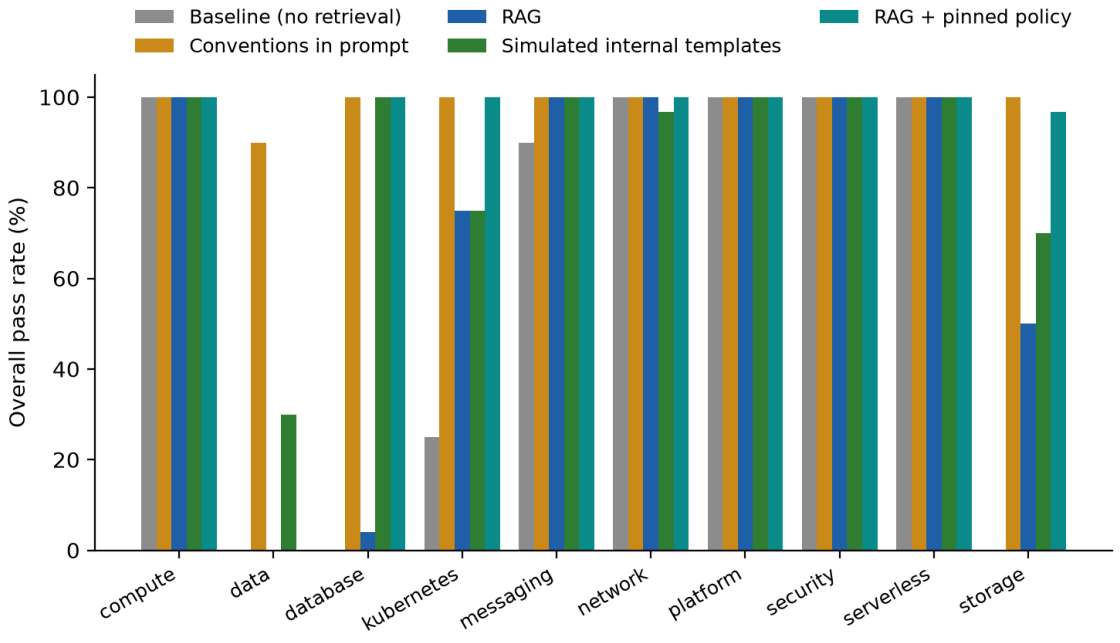


Figure 11: Overall pass rate by task category in the final run.

The categories with no applicable policy, or with policy the model already follows (compute, network, platform, security, serverless, messaging), pass at 90% or more in every arm. The arms differ in storage, database, Kubernetes and data. Pinning lifts database from 4% to 100%, storage from 50% to 97% and Kubernetes from 75% to 100%. It leaves data (BigQuery) at 0%, against 90% for the conventions arm: BigQuery has no policy page of its own to pin, and the general policy does not name the setting the BigQuery rule checks (Section 5.7.3).

5.5 Validation with the reviewed internal templates

The templates arm tests whether the conventions-in-prompt result depends on unusually clean, hand-written conventions. In the earlier run, scored against two policies, it did not: the templates and conventions arms were indistinguishable (overall pass 95.8% against 95.2%). In the final run, with the reviewed and corrected templates and five policies, the templates arm reaches 86.7% against 99.4%, and the conventions are better on 7 tasks and worse on none.

The templates arm's 22 failing samples contain: bucket retention not declared in 9; bigquery deletion setting not declared in 7; intent not met in 5; invalid configuration (init/validate) in 1. The bucket-retention and BigQuery failures follow from how the templates state the retention rule, as a one-line note ("buckets, cloud sql, bigquery -> need either deletion protection or a documented retention policy") without saying which arguments satisfy it, whereas the hand-written conventions show it in worked examples. The intent failures are not template failures at all: all of them are on task `gke-01`, where the templates arm built the correct three-node pool and the intent check rejected it (Section 5.7.5). With that check corrected, the templates arm passes 148 of 165 (90%) and the conventions arm 159 of 165 (96%), so the gap narrows but remains.

Effect of the review. The review's most serious finding, point-in-time recovery advised for MySQL (F1), had caused plan failures on task `sql-02` in the earlier run (simulated internal templates 4 of 5; conventions in prompt 4 of 5). After the correction, no sample in any arm of the final run failed with that error. The conclusion is that messier documentation costs some compliance, mainly where a rule is stated without showing how to meet it, and that defects in organisational documents pass straight into generated code, which is why the documents need review as much as the generator does.

5.6 Retrieval configuration experiments

Metric (RAG arm)	Default: 8 chunks, cap 3, chunk 1500	3 chunks	No per-document cap	Chunk size 800	Policy indexed per resource type
Samples	165	99	99	99	165
Overall pass	78.8% [72-84]	73.7% [64-81]	78.8% [70-86]	77.8% [69-85]	66.7% [59-73] *
pass@3 (overall)	n/a	78.8%	84.8%	n/a	n/a
Retrieval hit rate	97.0%	94.4%	97.0%	98.0%	97.0%
Policy retrieved	12%	3%	9%	3%	36%
Missing a label	20%	22%	19%	21%	19%

Table 17: Retrieval configurations of the RAG arm before the final run, scored against the earlier policy set. * Scored against all five policies, so its overall pass is not comparable with the other columns.

None of the generic retrieval settings changed the outcome. Three chunks instead of eight lowered the hit rate slightly, and removing the per-document cap or halving the chunk size left it unchanged; the overall pass intervals overlap in every case. In all four, the policy was retrieved in at most 12% of samples and about one sample in five was missing a label. These settings change how much provider documentation the model sees, not whether the policy is among it, because the requirement text does not resemble the policy text however many chunks are taken. Indexing the policy per resource type is the only setting that changed policy retrieval, tripling it, yet the share of samples missing a label hardly moved (20% with the default setting, 19% with per-resource indexing). Getting the policy retrieved more often was not enough on its own (Section 5.4).

Metric (RAG + pinned policy)	4 chunks	8 chunks (final run)	12 chunks
Samples	99	165	99
Functional pass	99.0% [94-100]	99.4% [97-100]	100.0% [96-100]
Overall pass	93.9% [87-97]	93.3% [88-96]	93.9% [87-97]
pass@1 (overall)	93.9%	93.3%	93.9%
pass@3 (overall)	97.0%	93.9%	93.9%
Retrieval hit rate	96.0%	97.0%	100.0%

Table 18: The policy-pinning arm with different numbers of retrieved chunks, scored against all five policies.

With the policy pinned, the number of retrieved chunks matters little. Overall pass ranges from 93.3% to 93.9% across four, eight and twelve chunks, and the retrieval hit rate from 96.0% to 100.0%, with overlapping intervals. Once the policy no longer has to win a place among the retrieved chunks, retrieval only has to find the provider documentation, which it does at every setting tried. The default of eight chunks is kept: it is not shown to be better than four or twelve, and neither alternative is shown to be better than it.

5.7 Detailed failure analysis

This section reviews every failing sample of the final run by root cause (Section 4.3.6): first where the failures are, then six cases, each examined with the configurations that produced it.

5.7.1 Where the failures are

Table 19 counts the failing samples of each arm by cause. A sample can have more than one.

Root cause	Baseline	Conventions in prompt	RAG	Simulated internal templates	RAG + pinned policy
Failing samples	81	1	54	22	11
Missing `environment` label	69	0	33	0	0
Public exposure	20	0	6	0	0
BigQuery deletion setting not declared	10	1	10	7	10
Bucket retention not declared	10	0	11	9	0
Other policy rule	0	0	1	0	0
Invalid configuration (init/validate)	2	0	2	1	1
Intent not met	4	0	0	5	1

Table 19: Failing samples by root cause in the final run (165 samples per arm).

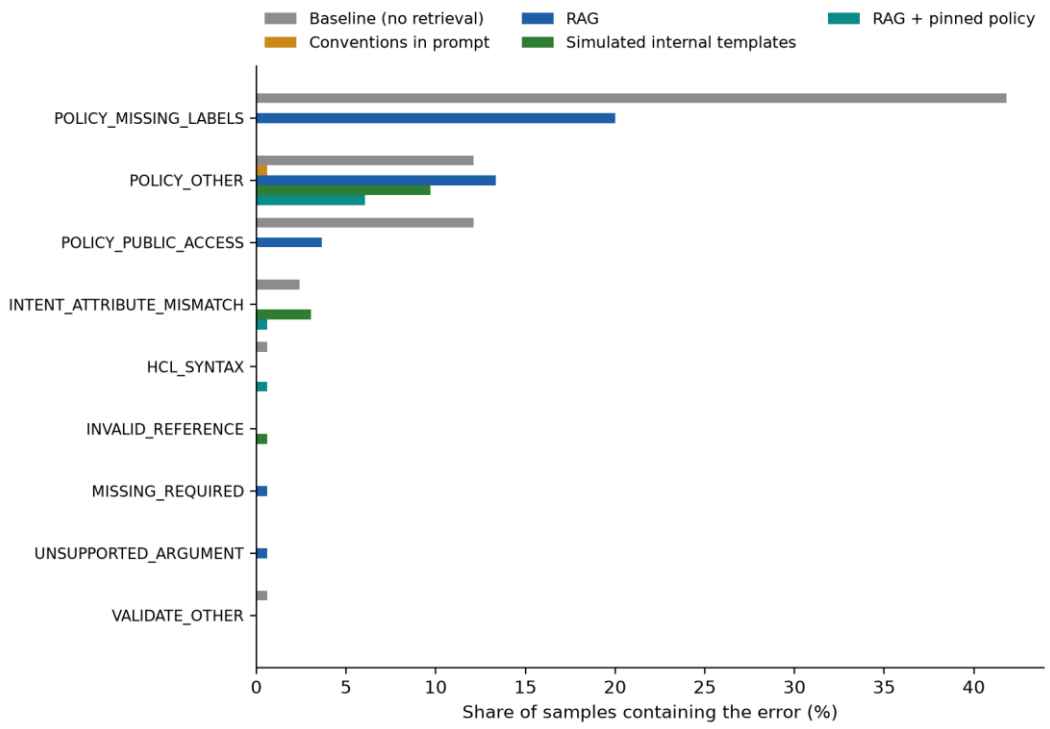


Figure 12: Share of samples containing each error category in the final run.

Almost every failure is a policy failure, and four root causes account for nearly all of them (Cases 1 to 4). Cases 5 and 6 cover the policy-conflict tasks and the few technical errors.

5.7.2 Case 1: unstated policy that did not reach the model

The most common RAG failure is a missing `environment` label (33 samples: 21 database, 7 storage, 5 kubernetes). The requirement never mentions labels, so the model labels a resource only if the policy is in its context and it treats the policy as binding. Cloud SQL is the clearest case: the database category passes at 4% for RAG and 100% with pinned policy. The Cloud SQL provider pages are long and detailed and fill the retrieved context, so even the resource-specific policy page was retrieved in only 5 of 25 (20%) RAG samples of the database tasks; when it is pinned, every sample carries the label. This is the failure the review's first request was about, and pinning removes it.

5.7.3 Case 2: a policy that does not say what is enforced

The two BigQuery tasks fail in most samples of every arm except the conventions arm. Samples failing the BigQuery retention rule: baseline 10 of 10; conventions in prompt 1 of 10; rag 10 of 10; simulated internal templates 7 of 10; rag + pinned policy 10 of 10. The policy text says that BigQuery datasets "must have deletion protection or an explicit retention policy". The Rego rule that enforces it checks one specific argument, `delete_contents_on_destroy`, which no organisational document mentions. Task `bq-01` asks for tables that expire after 30 days, and the configurations set `default_table_expiration_ms` accordingly, which a reader could fairly take as an explicit retention policy: there, the model arguably followed the policy as written and the check rejected it. Task `bq-02` asks for no retention at all, and nothing in the policy text tells the model which setting would satisfy the rule.

The conventions-arm samples that pass set `delete_contents_on_destroy = false` explicitly. That arm sees all four convention documents, whose examples for other resources declare their deletion settings explicitly, but the data do not show which part of the context prompted it. This is the information-gap failure described by Jouini [4]: the requirement cannot be met reliably from the text the model is given. The remedy is in the policy, not the generator: state the enforced argument in the policy text, or generate the text and the Rego rule from one source (Section 6.2).

5.7.4 Case 3: a defect in organisational documentation copied into code

In the earlier run, task `sql-02` (a highly available MySQL instance with backups) failed `terraform plan` with the error *"point_in_time_recovery_enabled is only available for the following [POSTGRES SQLSERVER]"* in simulated internal templates 4 of 5; conventions in prompt 4 of 5; rag 0 of 5; baseline 2 of 5 samples. Both the internal templates and the hand-written conventions advised point-in-time recovery for every engine. The model makes this

mistake from its own knowledge too, as the baseline shows, but with the flawed advice as its only guidance it made it in almost every sample. RAG retrieved the same flawed conventions page and also the provider page, whose MySQL example uses `binary_log_enabled`, and it followed the provider page every time. The review found the defect (F1, Section 4.3.7), both documents were corrected, and in the final run no sample fails this way. A model will reproduce a defect in the only guidance it has, so organisational documents are part of the system's attack and error surface, as Section 2.5 argued for poisoned documents.

5.7.5 Case 4: an intent check that rejected correct answers

Task `gke-01` asks for "a regional GKE cluster in asia-south1 ... plus a separately managed node pool of 3 e2-standard-4 nodes". On a regional node pool, `node_count` is the number of nodes *per zone*, and asia-south1 has three zones, so three nodes in total means `node_count = 1` (or `node_count = 3` with `node_locations` restricted to one zone). The task's intent check, written by the author, requires `node_count = 3`, which on a regional pool creates nine nodes. Reviewing the failures showed that the check has the answer backwards:

Arm	Pools with 3 nodes in total (correct)	Pools with 9 nodes	Passed the intent check
Baseline	3	2	2
Conventions in prompt	0	5	5
RAG	0	5	5
Simulated internal templates	5	0	0
RAG + pinned policy	0	5	5

Table 20: Node-pool sizes generated for `gke-01`, and which passed the intent check.

Every sample that passed built nine nodes, and every sample that built the requested three failed. The templates arm built three nodes in all five samples, because the reviewed GKE template now states that the count is per zone (finding F5); the other arms mostly set `node_count = 3`. Correcting the check after the fact would change the pre-specified result, so the headline figures keep the check as it was, and Table 21 gives the effect of correcting it.

Overall pass	Baseline	Conventions in prompt	RAG	Simulated internal templates	RAG + pinned policy
As specified (headline)	50.9%	99.4%	67.3%	86.7%	93.3%
With the `gke-01` check corrected	50.9%	96.4%	64.2%	89.7%	90.3%

Table 21: Sensitivity of the overall pass rate to the `gke-01` intent check.

The correction moves each arm by three percentage points at most and does not change the order of the arms that matter for the conclusions: pinned policy remains well above RAG and the conventions remain highest. It does bring the templates arm level with pinned policy. More importantly, it shows that an automated check written by hand can be wrong in a way no arm's pass rate reveals: the error was only visible because one arm consistently gave a different answer. The same applies to any organisation's policy checks.

5.7.6 Case 5: requests that conflict with policy

Metric	Baseline	Conventions in prompt	RAG	Simulated internal templates	RAG + pinned policy
Intent pass (did what was asked)	80.0% [49-94]	100.0% [72-100]	100.0% [72-100]	100.0% [72-100]	100.0% [72-100]
OPA violation rate	100.0% [68-100]	100.0% [72-100]	100.0% [72-100]	100.0% [72-100]	100.0% [72-100]

Table 22: The two tasks that explicitly request policy-forbidden configurations (a public website bucket and a Cloud SQL instance with a public IP).

When the requirement itself asks for what policy forbids, every arm does what was asked and violates policy, including the policy-pinning arm, whose prompt says the policy applies unless the requirement explicitly asks otherwise. No arm refused or warned. This is the intended behaviour of a generator that serves explicit requests, and it is why the validation layer is needed: the policy check is the only thing that stops a conflicting request from reaching review unflagged.

5.7.7 Case 6: residual technical errors, and the published taxonomy

Technical diagnostic	Baseline	Conventions in prompt	RAG	Simulated internal templates	RAG + pinned policy
init: Terraform encountered problems during initialisation, including proble	1	0	0	0	1
init: Unclosed template interpolation sequence	1	0	0	0	1
validate: Invalid resource type	1	0	0	0	0
validate: Missing required argument	0	0	1	0	0
validate: Unsupported argument	0	0	1	0	0
validate: Reference to undeclared resource	0	0	0	1	0

Table 23: Technical (non-policy, non-intent) diagnostics among the failing samples of the final run.

Technical errors are rare: a handful of samples across 825 headline samples fail `terraform init` or `validate`, for example for an unclosed template interpolation, an undeclared reference or an argument the provider does not support, and no sample failed `terraform plan` once the point-in-time-recovery defect was corrected. The counts are too small to compare arms.

Mapped onto the taxonomy of Nekrasov et al. [3], the technical categories correspond directly (HCL syntax to their syntax errors; argument, block, attribute and reference errors to their schema errors), but two mappings are not clean, and both are informative. Plan errors have no counterpart, because they catch provider-API constraints that a validate-time taxonomy cannot see. Policy violations have no counterpart either: the published taxonomy has no organisational-policy dimension, although that is where nearly every failure in this benchmark falls. The failure analysis adds a further distinction the taxonomy does not make, between failures of the generator and failures of the documents and checks it is evaluated against (Cases 2 to 4).

5.8 By difficulty

Difficulty	Baseline	Conventions in prompt	RAG	Simulated internal templates	RAG + pinned policy
Level 1	50%	100%	78%	96%	100%
Level 2	30%	98%	40%	84%	90%
Level 3	68%	100%	80%	82%	91%

Table 24: Overall pass rate by difficulty level in the final run.

Difficulty does not order the results: RAG passes 78% of level 1, 40% of level 2 and 80% of level 3 samples. As in the earlier run, most level-2 tasks are storage, database and Kubernetes tasks, where the policy applies, so difficulty and resource type are confounded in this task set and the category view of Section 5.4 is the more informative one.

5.9 Further checks from the earlier run

Three further checks were made in the earlier phase and are summarised here; their full results are in the mid-semester report. They were not repeated in the final run.

Near-greedy run. All arms of the earlier run were repeated once per task at temperature 0. The ordering and the size of the gaps were the same as at temperature 0.8 (overall pass: baseline 57.6%, RAG 78.8%, conventions 97.0%), so the findings are not an artefact of sampling.

Verifier-feedback refinement. Validator diagnostics from ten failures of the earlier run, covering every distinct combination of error codes, were fed back to the model for one retry, following [4]. Nine of ten were fully resolved. In two of the ten, the retry dropped the required provider version constraint while fixing the reported issue, because that constraint was not part of what the loop validated. A feedback loop can only protect what it checks.

Poisoning robustness. One adversarial document posing as an urgent policy update, instructing public read access on every bucket, was added to a copy of the knowledge base, following [7]. It was retrieved for five of six storage tasks in each of two runs, but in twelve generations the model never added the public grant. It did follow the poisoned default region on tasks that left the region unstated. An explicit requirement outweighed a retrieved document on the security-critical point; this is a small probe, not a robustness guarantee.

5.10 User evaluation: usability, completion time and developer trust

The user evaluation (objective O6) was run with 8 engineers from the delivery team: 4 with under one year of Terraform experience and 4 with two years or more. Each completed the same 4 tasks from the benchmark set (`gcs-01`, `gke-01`, `net-03`, `sql-03`), two with the assistant and two without it, in a counterbalanced within-subjects design (32 sessions in total). Task success was scored with the same automated validity and intent checks used in the benchmark. Time to completion was recorded per task. After the assistant condition each participant completed the System Usability Scale (SUS) [13] and a 12-item trust-in-automation questionnaire [14], followed by five semi-structured interview questions.

How the manual condition was actually run. The design called for the manual condition to use provider documentation and web search only. In practice, some participants used a general-purpose AI assistant during the manual condition. Use of other tools was not controlled or recorded per session, so it is not known which manual sessions were affected. The comparison below is therefore between *the project's assistant* and *participants' own unrestricted working practice, which for some included a general-purpose AI assistant*, not between the assistant and no assistant. This is the most important limitation of the user evaluation and is discussed further below.

Group	Condition	Tasks succeeded	Median time (s)	Mean time (s)
Under 1 year	Assistant	3 / 8	30	29
Under 1 year	Manual	3 / 8	61	78
2 years or more	Assistant	6 / 8	30	42
2 years or more	Manual	1 / 8	55	106
All participants	Assistant	9 / 16	30	36
All participants	Manual	4 / 16	59	92

Table 25: User evaluation: task success and time to completion by condition and experience level.

Task	Assistant: succeeded	Assistant: median s	Manual: succeeded	Manual: median s
gcs-01	1 / 4	25	0 / 4	78
gke-01	4 / 4	33	0 / 4	61
net-03	4 / 4	22	4 / 4	61
sql-03	0 / 4	34	0 / 4	54

Table 26: User evaluation results per task.

Measure (assistant vs manual)	Pairs	W+	p (exact, two-sided)	Rank-biserial r
Mean time per participant	8	3.0	0.039	-0.833
Success rate per participant	7	22.0	0.188	0.571

Table 27: Paired Wilcoxon signed-rank tests, one pair per participant (tied pairs dropped). A negative r means lower values with the assistant.

Completion time. Tasks done with the assistant took a median of 30 s against 59 s for the manual condition. 7 of 8 participants were faster on average with the assistant (Wilcoxon p = 0.039, r = -0.833). This is the clearest result of the user evaluation, but two qualifications apply. The absolute times are short for writing Terraform by hand, which is consistent with some manual sessions having used a general-purpose AI assistant, as described above. And with eight participants, the result shows a consistent direction, not a reliable estimate of the size of the saving.

Task success. 9 of 16 assistant sessions and 4 of 16 manual sessions passed the automated checks (per-participant Wilcoxon p = 0.188, not significant). The per-task table shows why this comparison is weak. Task `sql-03` was never completed successfully in either condition, so it contributes no information about the assistant. With only two counterbalancing sequences, each participant used the assistant on a fixed pair of tasks, so a participant's success difference largely reflects which tasks they had the assistant for, not the assistant alone. The success comparison is therefore reported but not interpreted as an effect.

Group	n	SUS, 0-100: mean (range)	Trust, 1-7: mean (range)
Under 1 year	4	69.4 (50.0-85.0)	5.75 (4.00-7.00)
2 years or more	4	58.1 (50.0-65.0)	5.27 (4.00-5.75)
All participants	8	63.8 (50.0-85.0)	5.51 (4.00-7.00)

Table 28: Usability (SUS) and trust in the assistant, collected after the assistant condition.

Usability and trust. The mean SUS score is 63.8, below the commonly used benchmark of 68 for an average system [16], with a wide spread (50.0 to 85.0). Mean trust in the assistant is 5.51 on the 1-7 scale, above the neutral midpoint of 4, and no participant scored it below 4.00. The two measures move together: the participants who rated usability highest also reported the most trust. With four participants per experience group, the difference between groups cannot be distinguished from individual variation. Trust was collected only after the assistant condition, so no comparison of trust between conditions is possible. The edit-count measure in the design (edits made to the assistant's output) was not recorded in any session and is not reported.

Interviews. The semi-structured interviews produced almost no usable data. Of the 8 participants, 2 gave any answer other than a blank or "NA", and those answers are single phrases (for example, that they would use the assistant "as it is reliable"). No thematic analysis is therefore possible, and the qualitative question of what makes developers trust or distrust generated infrastructure code remains open.

What the user evaluation does and does not show. It shows that the assistant can be used by engineers at both experience levels without training, that they completed tasks faster with it than with their usual practice, and that they reported moderate usability and above-neutral trust. It does not show an improvement in task success. It cannot attribute the time saving to the assistant rather than to AI assistance in general, because the manual condition was not controlled for AI use. It also gives no insight into why participants trusted the output. A repeat with controlled tool use, recorded edits, trust measured in both conditions, a Latin-square task assignment and interviews conducted live rather than by form is listed under Future Scope.

A caveat from the failure analysis. Task success in the user evaluation was scored with the benchmark's intent checks, including the `gke-01` check that rejects the correct three-node configuration (Section 5.7.5). The session records do not keep the submitted configurations, so the `gke-01` results cannot be re-scored. A participant who wrote the correct per-zone node count would have been scored as failing, so the 4 of 4 against 0 of 4 on that task may partly reflect the checker and not the condition.

5.11 Discussion

Answers to the research questions. For **RQ1**, retrieval has no measurable effect on technical correctness, because every arm is near ceiling on this task set. For **RQ2**, retrieval improves policy compliance over the baseline (50.9% to 67.3% overall pass), but most of the gain available comes from the policy reaching the model in a form it treats as binding: pinning it gives 93.3% and pasting all conventions 99.4%. For **RQ3**, nearly all errors are omissions of unstated policy; their root causes are policy that was not in the context, policy text that does not say what is enforced, a defect in the documents, and in one case the check itself. For **RQ4**, retrieval finds the provider documentation reliably at every setting tried (hit rate about 97%), generic settings do not change policy retrieval, per-resource indexing triples it, and with the policy pinned the amount of retrieved context hardly matters. For **RQ5**, engineers were faster with the assistant, rated its usability as moderate and trusted it above neutral, but the study cannot show a gain in task success.

Why pinning works and retrieval does not. Policy is cross-cutting: it applies to a request without resembling it. Similarity search is the wrong tool for selecting it, and even a retrieved policy chunk is presented as one reference passage among several. Selecting policy by rule from what was retrieved, and presenting it as mandatory, solves both problems while keeping

the prompt short. The conventions-in-prompt arm remains slightly better on this task set because it gives the model every convention and every worked example, which on the BigQuery tasks led it to declare the deletion setting explicitly, though the data do not show which part of the context did so. That advantage holds only while the conventions are short enough to paste whole; pinning is the design that scales.

What an organisation should take from this. The quality of the written policy matters as much as how it is delivered. A rule that names the setting it checks is followed; a rule written as a principle is interpreted, and the interpretation may not be the one the check enforces. Organisational documents need the same review as code, because the model copies their defects. And automated policy checks remain necessary in every configuration tested, both because no arm reaches 100% and because every arm builds what it is asked for even when the request breaks policy.

Threats to validity. These are listed in Section 4.3.11. The ones that matter most for this chapter are that the policy-pinning arm was designed after seeing the earlier run's results, on the same task set; that the conventions, templates and policies were written by the author; that the failure analysis found one defective intent check, which means others could exist; and that the user evaluation's manual condition was not controlled for AI use. With one model, 33 headline tasks and five comparisons, the category-level and paired results are best read as consistent directions with stated uncertainty.

CHAPTER 6 – CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This dissertation built a RAG assistant that generates Terraform for Google Cloud from natural-language requirements and validates every output with Terraform, Open Policy Agent policies and intent checks. It was evaluated in a final benchmark of 35 tasks, five arms and 875 samples, in six further retrieval configuration runs, and with eight engineers. Every sample is stored with its retrieved context and diagnostics, so every figure in this report can be traced back to the generated code.

Technical correctness was not the problem. The model wrote configurations that validate, plan and do what was asked in 96% to 100% of samples in every arm, with or without retrieval. On this task set, the value of grounding is not in fixing malformed code.

Policy was the problem, and getting policy to the model was most of the solution. Without any organisational context, half of all configurations broke policy. Retrieval helped (overall pass 50.9% to 67.3%) but left a large gap, because policy applies to every request without resembling any of them, so similarity search ranks it poorly. Indexing the policy per resource type tripled how often it was retrieved, but a retrieved policy chunk was still often not followed. Pinning the relevant policy pages in a mandatory prompt section closed most of the gap (93.3%), without pasting every document into the prompt. Pasting all the conventions in did slightly better again (99.4%), which is the practical choice while conventions are short.

The remaining failures point at the documents and the checks, not the model. A rule written in prose that does not name the setting its enforcement checks (BigQuery retention) was missed in most samples of every arm that had only the prose. A defect in an internal template (point-in-time recovery for MySQL) was copied into generated code until the template

was corrected. And one intent check rejected the correct answer and accepted a wrong one. For an organisation, this means that the quality of its written conventions and of its policy checks matters as much as the generator, and that automated policy validation remains essential: every arm built what was asked even when the request itself broke policy.

Engineers were faster with the assistant and moderately satisfied with it. In the user evaluation, seven of eight participants completed tasks faster with the assistant, and the median time roughly halved. Usability was moderate (SUS 63.8) and trust above neutral. The comparison was weakened by uncontrolled AI use in the manual condition and by the small sample, so it shows a consistent direction rather than a reliable size of effect.

6.2 Future scope

- **Independent review of the internal templates.** Have the organisation's platform team review the templates and conventions against actual practice, which the technical self-review could not establish (Section 4.3.7).
- **A controlled repeat of the user evaluation.** Control and record tool use in the manual condition, record edits to the assistant's output, measure trust in both conditions, use a Latin-square task assignment, and conduct the interviews live (Section 5.10).
- **Policy written for enforcement.** Generate the policy pages and the Rego rules from one source, so that every enforced setting is named in the text the model sees (Section 5.7.3).
- **Policy pinning without a hand-written mapping.** Select policy pages from the resource types named in the request or in a first draft of the configuration, so that new resource types are covered automatically.

- **Verifier feedback at scale.** Run the verifier-feedback arm, already implemented, as a full benchmark arm combined with policy pinning; the earlier probe resolved nine of ten failures in one retry (Section 5.9).
- **Broader evaluation.** Further models, more tasks per category (including more data and analytics tasks), a second cloud provider, and apply-time validation in a sandbox project to catch errors that only the cloud API detects.
- **Corrected intent checks.** Correct the `gke-01` intent check to count nodes across zones and audit the other multi-zone checks in the same way (Section 5.7.5).

6.3 Plan of work as completed

The phases below are those of the approved outline.

Sl No	Phases	Start Date - End Date	Work done	Status
1	Literature review and problem formulation	25 Jul 2026 - 14 Aug 2026	Review of LLM code generation, RAG and IaC benchmarks; problem formulated	COMPLETED
2	Abstract / outline submission and evaluation	15 Aug 2026 - 04 Sep 2026	Dissertation outline prepared and submitted	COMPLETED
3	RAG pipeline and knowledge-base design	05 Sep 2026 - 25 Sep 2026	Cloud, LLM, embedding model and vector store chosen; ingestion, retrieval, generation and validation designed	COMPLETED
4	Prototype development and mid-semester evaluation	26 Sep 2026 - 16 Oct 2026	Working prototype on Vertex AI, first full benchmark, mid-semester report	COMPLETED
5	Full implementation and validation tooling	17 Oct 2026 - 06 Nov 2026	Validation tooling, control and templates arms, retrieval ablations, near-greedy run, error analysis, verifier feedback, poisoning check, policy pinning, template review	COMPLETED
6	Benchmarking, user evaluation and final report	07 Nov 2026 - 30 Nov 2026	Final benchmark and retrieval experiments, user evaluation with eight engineers, final report	COMPLETED

Table 29: Plan of work and status.

APPENDIX A. TASK CATALOGUE

All tasks in the benchmark. Difficulty is 1 (single resource, simple) to 3 (several related resources).

ID	Category	Diff.	Requirement
gcs-01	storage	1	A Cloud Storage bucket named acme-invoices-archive in asia-south1 with object versioning enabled.
gcs-02	storage	2	A Cloud Storage bucket with a lifecycle rule that moves objects to NEARLINE storage after 30 days.
gcs-03	storage	2	A Cloud Storage bucket with uniform bucket-level access enabled and public access prevention enforced.
gcs-04	storage	1	A Cloud Storage bucket in europe-west1 using the COLDLINE storage class.
gcs-05	storage	1	A Cloud Storage bucket that automatically deletes objects 90 days after creation.
gcs-06	storage	3	A private Cloud Storage bucket and a service account that has read-only object access (roles/storage.objectViewer) to that bucket.
sql-01	database	1	A PostgreSQL 15 Cloud SQL instance using the db-custom-2-8192 machine tier.
sql-02	database	2	A highly available (regional) MySQL 8.0 Cloud SQL instance with automated backups enabled.
sql-03	database	2	A PostgreSQL Cloud SQL instance with no public IP address and deletion protection enabled.
sql-04	database	3	A PostgreSQL Cloud SQL instance with a database named appdb and a user named appuser.
sql-05	database	2	A Cloud SQL PostgreSQL instance with point-in-time recovery enabled.
gke-01	kubernetes	3	A regional GKE cluster in asia-south1 that removes the default node pool, plus a separately managed node pool of 3 e2-standard-4 nodes.
gke-02	kubernetes	2	A private GKE cluster (private nodes) with Workload Identity enabled.
gke-03	kubernetes	1	A GKE Autopilot cluster in asia-south1.
gke-04	kubernetes	2	A GKE node pool for an existing cluster named main-cluster in asia-south1, autoscaling between 1 and 5 nodes.
net-01	network	1	A custom-mode VPC network (no automatically created subnets).
net-02	network	3	A custom VPC network with one subnet using the range 10.10.0.0/24 in asia-south1.
net-03	network	2	A firewall rule that allows inbound TCP port 22 only from 203.0.113.0/24.
net-04	network	3	A Cloud Router and a Cloud NAT in asia-south1 so that private VMs can reach the internet.
net-05	network	1	A reserved global static external IP address.
vm-01	compute	2	A Compute Engine VM named web-1 of machine type e2-medium in zone asia-south1-a using the boot disk image debian-cloud/debian-12.
vm-02	compute	3	A Compute Engine VM with no external IP address that runs as a dedicated service account.
run-01	serverless	1	A Cloud Run v2 service in asia-south1 running the container image us-docker.pkg.dev/cloudrun/container/hello.
run-02	serverless	3	A Cloud Run v2 service that only the service account invoker-sa@my-project.iam.gserviceaccount.com may invoke (no public access).
ps-01	messaging	3	A Pub/Sub topic named orders with a pull subscription named orders-sub that has a 30 second acknowledgement deadline.
ps-02	messaging	3	A Pub/Sub subscription with a dead-letter topic that gives up after 5 delivery attempts.
sch-01	serverless	3	A Cloud Scheduler job that runs every day at 02:00 and publishes a message to a Pub/Sub topic.
bq-01	data	2	A BigQuery dataset named analytics in asia-south1 whose tables expire after 30 days by default.
bq-02	data	3	A BigQuery dataset and a table named events in it with a schema of timestamp (TIMESTAMP) and user_id (STRING).
sm-01	security	1	A Secret Manager secret named db-password with automatic replication.
kms-01	security	3	A Cloud KMS key ring and a symmetric crypto key in it that rotates automatically every 90 days.

ID	Category	Diff.	Requirement
ar-01	platform	1	An Artifact Registry Docker repository named app-images in asia-south1.
dns-01	network	3	A private Cloud DNS managed zone for internal.example.com. that is visible only to a VPC network.
pol-01	policy_conflict	2	A Cloud Storage bucket for hosting a public static website that anyone on the internet can read.
pol-02	policy_conflict	2	A MySQL Cloud SQL instance with a public IP address, because our office VPN cannot use the Cloud SQL Auth Proxy.

Table 30: The 35 benchmark tasks.

APPENDIX B. PROMPT TEMPLATES

B.1 Retrieval-augmented arm: system prompt

```
You are an infrastructure engineer generating Terraform for Google Cloud Platform (the `google`/`google-beta`
providers only).
Use the retrieved documentation and best-practice snippets below as your source of truth for resource names,
arguments, and defaults — prefer them over your own memory of the provider, since they may be more current.
If the retrieved context does not cover something you need, say so in a comment rather than guessing.


Output a single Terraform configuration (HCL).
Do not include prose outside of HCL comments.
Include a `terraform { required_providers { google = ...
} }` block and a `provider "google"` block.
Give every resource sensible labels/names and follow least-privilege / secure defaults (e.g.
no public access unless explicitly requested).
Target provider: `hashicorp/google` with version constraint `~> 8.0` (use exactly this constraint in
required_providers).
```

B.2 Baseline arm: system prompt

```
You are an infrastructure engineer generating Terraform for Google Cloud Platform (the `google`/`google-beta`
providers only), from memory only — no reference documentation is provided.
Output a single Terraform configuration (HCL) with no prose outside of HCL comments.
Target provider: `hashicorp/google` with version constraint `~> 8.0` (use exactly this constraint in
required_providers).
```

B.3 User prompt templates

```
Retrieved context:
{context}


Requirement:
{requirement}


Generate the Terraform configuration.


----


Requirement:
{requirement}


Generate the Terraform configuration.
```

B.4 Policy-pinning arm: system prompt and user template

```
You are an infrastructure engineer generating Terraform for Google Cloud Platform (the `google`/`google-beta`
providers only).
Use the retrieved documentation and best-practice snippets below as your source of truth for resource names,
arguments, and defaults — prefer them over your own memory of the provider, since they may be more current.
If the retrieved context does not cover something you need, say so in a comment rather than guessing.


Output a single Terraform configuration (HCL).
Do not include prose outside of HCL comments.
Include a `terraform { required_providers { google = ...
} }` block and a `provider "google"` block.
```

```
Give every resource sensible labels/names and follow least-privilege / secure defaults (e.g.
no public access unless explicitly requested).
Target provider: `hashicorp/google` with version constraint `~> 8.0` (use exactly this constraint in
required_providers).
The organisational policy section is mandatory, not reference material: apply every rule in it that applies to
the resources you generate (labels, region, public access, retention, roles, secrets), unless the requirement
explicitly asks for something different.

---

Organisational policy (mandatory):
{policy}

Retrieved context:
{context}

Requirement:
{requirement}

Generate the Terraform configuration.
```

APPENDIX C. ORGANISATIONAL POLICY RULES

Rego rules evaluated by Conftest against every generated configuration. All of them were in force for the final run, `final-01`; the earlier run, `main-01`, was scored against the first four only.

Rule	What it forbids
Required labels	A Cloud Storage bucket, Cloud SQL instance or GKE cluster without an `environment` label.
Public IAM grants	Any `*_iam_member` or `*_iam_binding` granting access to `allUsers` or `allAuthenticatedUsers`.
Cloud SQL exposure	A Cloud SQL instance with a public IP, or an authorised network of `0.0.0.0/0`.
Open ingress	An ingress firewall rule open to `0.0.0.0/0`.
Broad service-account roles	`roles/owner` or `roles/editor` granted at project level to a service account.
Retention/protection declared	A Cloud Storage bucket, Cloud SQL instance or BigQuery dataset with no explicit retention or deletion-protection setting.
No hardcoded secrets	A literal password, private key, API key, client secret or access token as a resource argument, instead of a reference.

Table 31: Policy rules.

APPENDIX D. RUN MANIFEST OF THE REPORTED BENCHMARK

The recorded settings of the run whose results are reported in Section 5 (`final-01`).


```
{
  "run_id": "final-01",
  "label": "final run: 5 arms incl. rag_policy; self-reviewed templates; corrected Cloud SQL KB doc; all 5
policies",
  "started_at": "2026-11-03T17:32:33.896800+00:00",
  "git_commit": "a63a7f4922f025c16535a097f9d7af94a695dbe5",
  "git_dirty": true,
  "mode": "gcp",
  "generation_model": "gemini-3.8-flash",
  "generation_location": "global",
  "temperature": 0.8,
  "embedding_model": "text-embedding-005",
  "chunk_size": 1500,
  "chunk_overlap": 150,
  "top_k": 8,
  "max_chunks_per_doc": 3,
  "index_chunks": 1053,
  "provider_docs_tag": "v8.3.0",
  "target_provider_constraint": "~> 8.0",
  "terraform_version": "1.16.2",
  "conftest_version": "Conftest: 0.70.0",
  "task_file": "data\\eval\\gcp_tasks.jsonl",
  "task_file_sha256": "b4f16513190e9bcac0b1caac3aae01fc09bd0cceae295cb42dd3bada5af9dad4",
  "policy_sha256": "4c0c23007ac4fa66c6d4ed55fac65bb0343a948741a2884f17ce31752908ac74",
  "arms": [
    "baseline",
    "conventions",
    "rag",
    "templates",
    "rag_policy"
  ],
  "samples_per_task": 5,
  "terraform_plan_run": true,
  "python": "3.14.4"
}
```

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[2] P. T. J. Kon et al., "IaC-Eval: A Code Generation Benchmark for Cloud Infrastructure-as-Code Programs," NeurIPS 2024 (Datasets and Benchmarks). Code and data: https://github.com/autoiac-project/iac-eval.

[3] R. Nekrasov, S. Fossati, I. Kumara, D. A. Tamburri and W.-J. van den Heuvel, "IaC Generation with LLMs: An Error Taxonomy and A Study on Configuration Knowledge Injection," arXiv:2512.14792, 2025.

[4] M. Jouini, "Verifier-First Evaluation of Agentic LLMs for Infrastructure-as-Code Generation," arXiv:2607.20478, 2026.

[5] S. Davidson, L. Sun, B. Bhasker, L. Callot and A. Deoras, "Multi-IaC-Eval: Benchmarking Cloud Infrastructure as Code Across Multiple Formats," arXiv:2509.05303, 2025.

[6] F. L. S. Vargas, R. B. Mansilha and D. Kreutz, "Security-First Evaluation of Text-to-Terraform: Benchmarking LLMs and SLMs for Secure IaC Generation," arXiv:2608.02672, 2026.

[7] K. Ye, L. Su and C. Qian, "ImportSnare: Directed 'Code Manual' Hijacking in Retrieval-Augmented Code Generation," arXiv:2509.07941, 2025.

[8] A. A. Khan, M. T. Hasan, K. K. Kemell, J. Rasku and P. Abrahamsson, "Developing Retrieval Augmented Generation (RAG) based LLM Systems from PDFs: An Experience Report," arXiv:2410.15944, 2024.

[9] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[10] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," Journal of the American Statistical Association, vol. 22, no. 158, pp. 209-212, 1927.

[11] HashiCorp, "Terraform Provider for Google Cloud Platform, release v8.3.0 documentation," https://github.com/hashicorp/terraform-provider-google (MPL-2.0).

[12] Open Policy Agent, "Policy Language (Rego)" and Conftest documentation, https://www.openpolicyagent.org and https://www.conftest.dev.

[13] J. Brooke, "SUS: A quick and dirty usability scale," in Usability Evaluation in Industry, Taylor & Francis, 1996.

[14] J.-Y. Jian, A. M. Bisantz and C. G. Drury, "Foundations for an empirically determined scale of trust in automated systems," International Journal of Cognitive Ergonomics, vol. 4, no. 1, pp. 53-71, 2000.

[15] Google Cloud, "Generative AI on Vertex AI: locations and model availability," https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/locations (accessed September 2026).

[16] J. Sauro, A Practical Guide to the System Usability Scale: Background, Benchmarks & Best Practices, Measuring Usability LLC, Denver, 2011.

GLOSSARY

Technical terms used in this report. Abbreviations are listed separately.

Term	Meaning
Arm	One generation strategy compared in the benchmark (baseline, RAG, conventions-in-prompt, templates, RAG with pinned policy).
Chunk	A fragment of a knowledge-base document, embedded and stored as one unit for retrieval.
Conftest	A command-line tool that evaluates Open Policy Agent (Rego) policies against configuration files.
Embedding	A numeric vector representing the meaning of a text, used to find similar texts.
Functional pass	A sample that passes terraform validate, terraform plan and the task's intent check.
Hit rate (retrieval)	The share of resource types a task needs whose documentation appears in the retrieved chunks.
Infrastructure as Code (IaC)	Describing cloud infrastructure in version-controlled declarative files instead of manual set-up.
Intent check	An automated check that a configuration contains the resources and settings the requirement asked for.
Knowledge base	The documents the assistant retrieves from: pinned provider documentation and organisational conventions.
Large language model (LLM)	A neural network trained on large text corpora that generates text, including code, from a prompt.
Open Policy Agent (OPA)	A general-purpose policy engine; policies are written in the Rego language.
Overall pass	A functional pass that also has no organisational-policy violation.
pass@k	The probability that at least one of k samples for a task passes, estimated without bias from n >= k samples.
Pinned policy	Organisational policy text placed in a fixed, mandatory section of the prompt rather than ranked by retrieval.
Policy as code	Organisational rules expressed as machine-checkable policies (here, Rego) rather than prose.
Provider (Terraform)	The plugin that maps Terraform resources onto a cloud's API; here hashicorp/google.
Retrieval-Augmented Generation (RAG)	Generating text with retrieved documents placed in the model's context.
Sign test	A non-parametric test of whether one arm wins more paired comparisons than the other.
System Usability Scale (SUS)	A 10-item questionnaire giving a usability score from 0 to 100.
terraform plan	Terraform's dry run, which checks a configuration against the provider and the cloud API without changing anything.
terraform validate	Terraform's static check of syntax and provider schema.
Vertex AI	Google Cloud's managed machine-learning platform, used here for embeddings and Gemini generation.
Wilcoxon signed-rank test	A non-parametric test for paired measurements, used here for the user evaluation.
Wilson interval	A confidence interval for a proportion that behaves well for small samples and extreme rates.

Table 32: Glossary of technical terms.

ABBREVIATIONS

Abbreviation	Expansion
ADC	Application Default Credentials
API	Application Programming Interface
CI	Confidence Interval
GCP	Google Cloud Platform
GCS	Google Cloud Storage
GKE	Google Kubernetes Engine
HCL	HashiCorp Configuration Language
IaC	Infrastructure as Code
KB	Knowledge Base
LLM	Large Language Model
OPA	Open Policy Agent
RAG	Retrieval-Augmented Generation
Rego	The policy language used by Open Policy Agent
RQ	Research Question
SUS	System Usability Scale
TF-IDF	Term Frequency - Inverse Document Frequency
WILP	Work Integrated Learning Programmes

Table 33: Abbreviations used in this report.